MAIE 5532: Machine Learning Systems

Week 9 - Pruning and Sparsity

Program: MSc in AI & Entrepreneurship

Term: Fall 2026/27

Instructor: Lefteris NTAFLOS, PhD

The Hong Kong University of Science and Technology

Week 9 Overview

Pruning and Sparsity: Optimizing Models Through Strategic Parameter Removal

Learning Objectives:

- Master pruning techniques for reducing model complexity while preserving accuracy
- Analyze the trade-offs between unstructured, structured, and dynamic pruning approaches
- Design pruning strategies for different deployment scenarios and constraints
- Implement practical pruning workflows using modern frameworks

Required Readings:

- Book 1 – Volume 1: Model Compression
- Book 1 – Volume 1: Hardware Acceleration
- Book 1 – Volume 1: Benchmarking
- Book 1 – Volume 1: Model Serving
- Book 2: Chapter 15: Optimizing Latency
- Book 2: Chapter 17: Optimizing Model and Binary Size



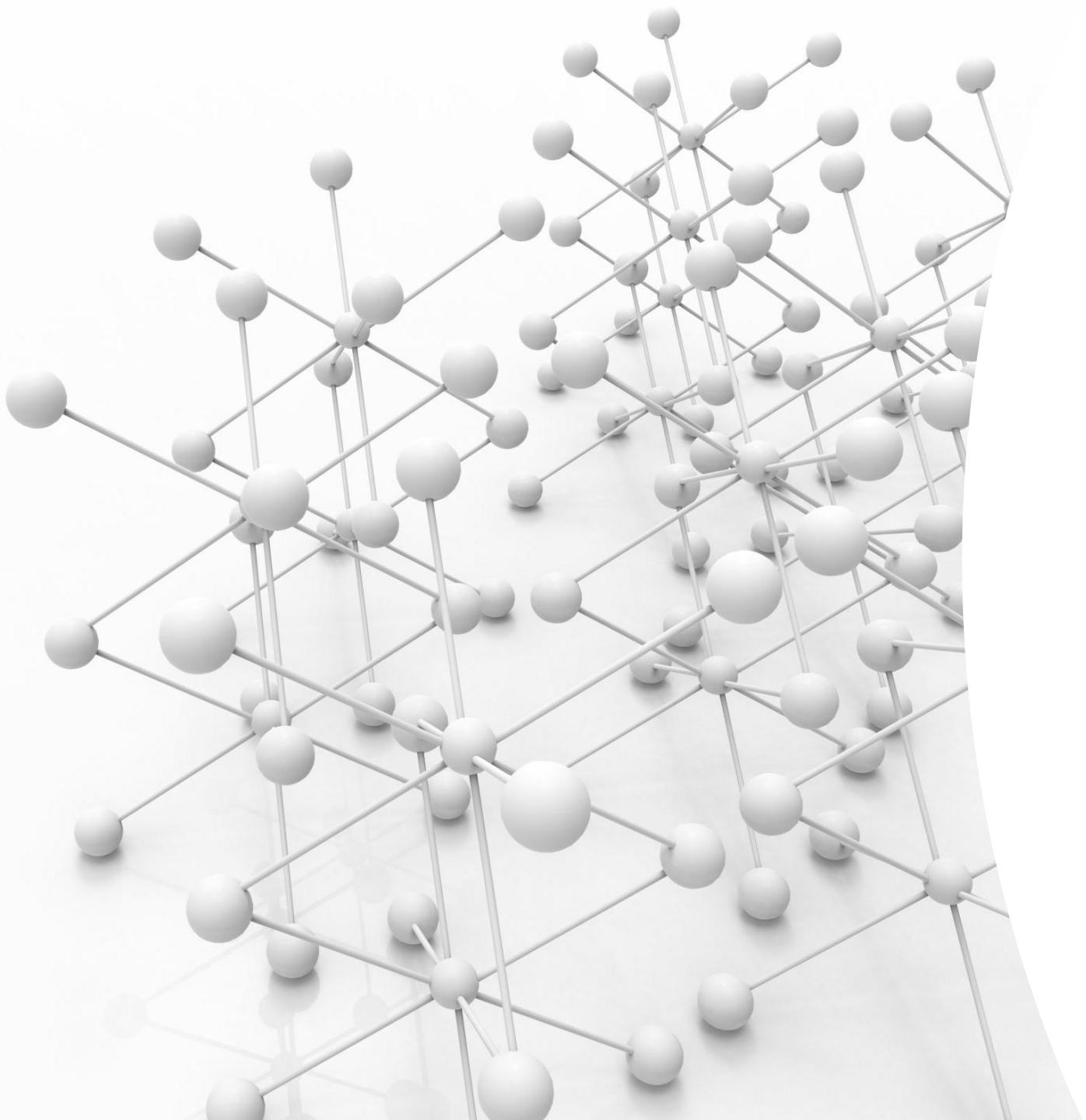
The Overparameterization Problem

Why Modern Neural Networks Need Pruning

State-of-the-art neural networks contain millions or billions of parameters, yet research consistently shows that many of these parameters contribute minimally to final predictions. This overparameterization creates three critical challenges:

- **Memory Requirements:** Large models demand substantial storage, creating bottlenecks in distributed ML pipelines and limiting deployment on resource-constrained devices like smartphones and IoT systems
- **Computational Cost:** More parameters directly translate to higher inference latency and energy consumption, particularly problematic for real-time applications such as autonomous systems and mobile AI
- **Scalability Limitations:** Training and deploying large models requires extensive compute, memory, and power resources, increasing operational costs and limiting accessibility

The key insight driving pruning research is that not all parameters are equally important. Many weights contribute little to decision-making and can be removed without substantial performance degradation.



Session Structure

Today's Organization

Session 1: Pruning Fundamentals and Mathematics
- Definition and motivation for pruning
- Mathematical formulation of the pruning problem
- Unstructured vs structured pruning approaches
- Target structures: neurons, channels, and layers

Session 2: Pruning Strategies and Implementation
- Iterative vs one-shot pruning workflows
- Lottery Ticket Hypothesis insights
- Framework implementation using PyTorch and TensorFlow
- Hardware compatibility considerations

Session 3: Advanced Techniques and Deployment
- Dynamic pruning for adaptive systems
- Size-performance trade-offs in embedded contexts
- Real-world deployment considerations
- Integration with other optimization techniques

SESSION 1: PRUNING FUNDAMENTALS AND MATHEMATICS

Pruning: The Main Goal

Pruning reduces the number of active weights while trying to preserve model accuracy.

$$\min_{\widetilde{W}} L(\widetilde{W}) \text{ subject to } \| \widetilde{W} \|_0 \leq k$$

Symbol	**Meaning**
$L(\widetilde{W})$	prediction loss / error
$\| \widetilde{W} \|_0$	number of nonzero weights
k	maximum weights allowed

For example, if the original network has 1 million weights and we choose $k = 200{,}000$, we are asking: **which 200,000 weights should we keep so that accuracy changes as little as possible?**

Why Is This Difficult?

Choosing the "best" weights is hard

$\| \widetilde{W} \|_0 \leq k$ requires selecting a subset of weights.

For large models:

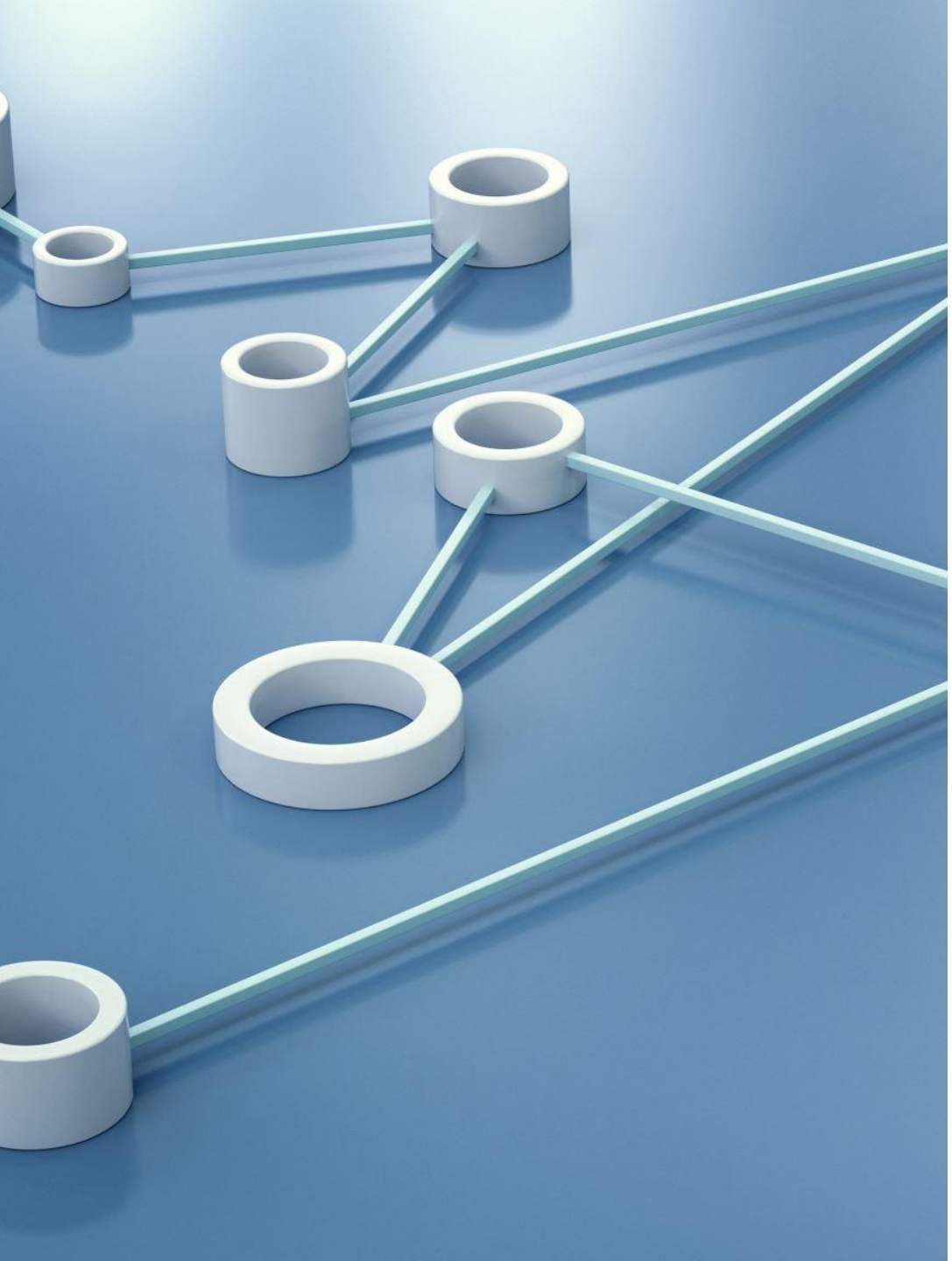
- enormous number of possible subsets
- exact search is impractical

Common relaxation

$$\min_{\widetilde{W}} \left[L\left(\widetilde{W}\right) + \lambda \| \widetilde{W} \|_1 \right]$$

$$\| \widetilde{W} \|_1 = \sum_i | \widetilde{w}_i |$$

A **small** λ prioritizes accuracy.
A **large** λ puts more pressure on weights to become small.



Definition of Pruning

Core Concept and Motivation

Pruning is a model optimization technique that removes unnecessary parameters from a neural network while maintaining predictive performance. By systematically eliminating redundant weights, neurons, or layers, pruning reduces model size and computational cost.

- **Memory Efficiency:** Directly addresses storage constraints by reducing parameter count, making models feasible for deployment on resource-constrained devices
- **Computational Savings:** Fewer parameters translate to reduced arithmetic operations during inference, improving latency and energy efficiency
- **Preserved Architecture:** Unlike architectural changes, pruning maintains the fundamental network structure while optimizing its parameter usage
- **Deployment Enablement:** Makes sophisticated models viable for edge computing, mobile devices, and embedded systems with limited resources
- **Systematic Approach:** Provides principled methods for identifying and removing redundant parameters based on various importance metrics



Pruning Challenges in ML Systems

From Theory to Practice

The gap between training performance and deployment efficiency creates several challenges that pruning addresses:

- **High Memory Requirements:** Large models require substantial storage, limiting feasibility on smartphones, IoT devices, and embedded systems where every kilobyte matters
- **Increased Computational Cost:** More parameters lead to higher inference latency and energy consumption, problematic for real-time applications and battery-powered devices
- **Scalability Limitations:** Training and deploying large models at scale is resource-intensive, requiring high-bandwidth communication and storage infrastructure
- **Hardware Utilization:** Modern accelerators are optimized for specific computation patterns; sparse models may not fully utilize available hardware without specialized support
- **Performance vs Efficiency Trade-offs:** Finding the optimal balance between model accuracy and resource consumption requires systematic optimization approaches

Pruning Mathematics: The Hard Problem

The **ℓ_0-norm** is simply a count of the number of non-zero values in a set. In pruning, it's used to measure sparsity by counting how many weights in the network have been set to zero.

Why Finding Optimal Sparsity is Intractable

The discrete nature of the ℓ_0-norm constraint makes exact pruning optimization computationally infeasible:

- **Exponential Complexity:** Finding the optimal subset of parameters to retain requires evaluating 2^n possible configurations for n parameters
- **Combinatorial Explosion:** Deep networks with millions of parameters make exhaustive search impossible within reasonable time constraints
- **Binary Decision Problem:** Each parameter must be either kept (1) or removed (0), creating a discrete optimization landscape without smooth gradients
- **No Polynomial Solution:** The problem belongs to NP-hard complexity class, meaning no known polynomial-time algorithm exists for optimal solutions
- **Practical Necessity:** Heuristic approaches and relaxations become essential for making pruning computationally tractable in real-world applications

Practical Relaxation Strategies

Making Pruning Computationally Feasible

Since exact optimization is intractable, practical methods use approximations and heuristics:

- **ℓ_1-Norm Regularization:** Replace discrete constraint with continuous penalty $\lambda||W||_1$ that encourages but doesn't enforce sparsity
- **Magnitude-Based Heuristics:** Assume larger weights are more important and remove smallest magnitude parameters iteratively
- **Iterative Approaches:** Gradually remove parameters in successive steps followed by fine-tuning to recover lost accuracy
- **Gradient-Based Importance:** Use training dynamics and gradient information to estimate parameter contributions to loss reduction
- **Activation-Based Metrics:** Evaluate parameter importance based on runtime activation patterns across representative data
- **Structured Approximations:** Focus on removing entire computational units rather than individual weights to simplify the optimization landscape

Binary Mask Formulation

A mask decides which weights stay active:

$$M \in \{0,1\}^{m \times n}$$

$$\widetilde{W} = M \odot W$$

- $M_{ij} = 1$: keep W_{ij}
- $M_{ij} = 0$: remove W_{ij}

$\odot$: element-wise multiplication

$$W = \begin{bmatrix} 0.8 & 0.1 \\ -0.4 & 0.02 \end{bmatrix}, M = \begin{bmatrix} 1 & 0 \\ 1 & 0 \end{bmatrix}$$

The weight matrix keeps the **same dimensions**, but some individual values become zero.

$$\widetilde{W} = \begin{bmatrix} 0.8 & 0 \\ -0.4 & 0 \end{bmatrix}$$

Creating the Mask

Magnitude-Based Pruning

Keep weights larger than a threshold τ:

$$M_{ij} = \begin{cases} 1, & |\ W_{ij}\ | > \tau \\ 0, & otherwise \end{cases}$$

Example: $\tau = 0.1$

Weight	Decision
0.8	keep
-0.4	Keep
0.02	remove

Small magnitude ⇒ more likely to be pruned

Result: A Sparse Model

Unstructured pruning creates sparsity

$$\widetilde{W} = M \odot W$$

- individual weights become zero
- layer shapes do not change
- many zeros ⇒sparse matrix

Typical workflow

Train → Prune → Fine−tune

Goal: fewer active weights, similar accuracy

Weight Matrix Transformation

Before and After Pruning Effects

Pruning transforms dense weight matrices into sparse representations with explicit zero values:

- **Dense to Sparse:** Original matrices with all non-zero values become sparse with many explicit zeros
- **Memory Implications:** Sparse matrices can use compressed storage formats but may not reduce actual memory without specialized representations
- **Computational Impact:** Sparse matrices require specialized kernels to exploit zero values for computational speedup
- **Hardware Considerations:** Standard dense matrix libraries cannot take advantage of sparsity without modification
- **Storage Efficiency:** Compressed sparse formats can significantly reduce storage requirements for high sparsity levels
- **Access Patterns:** Irregular sparsity patterns can hurt cache performance due to unpredictable memory access patterns

Unstructured Pruning: Individual Weight Removal

Fine-grained Parameter Elimination

Unstructured pruning removes individual weights while preserving the overall network architecture:

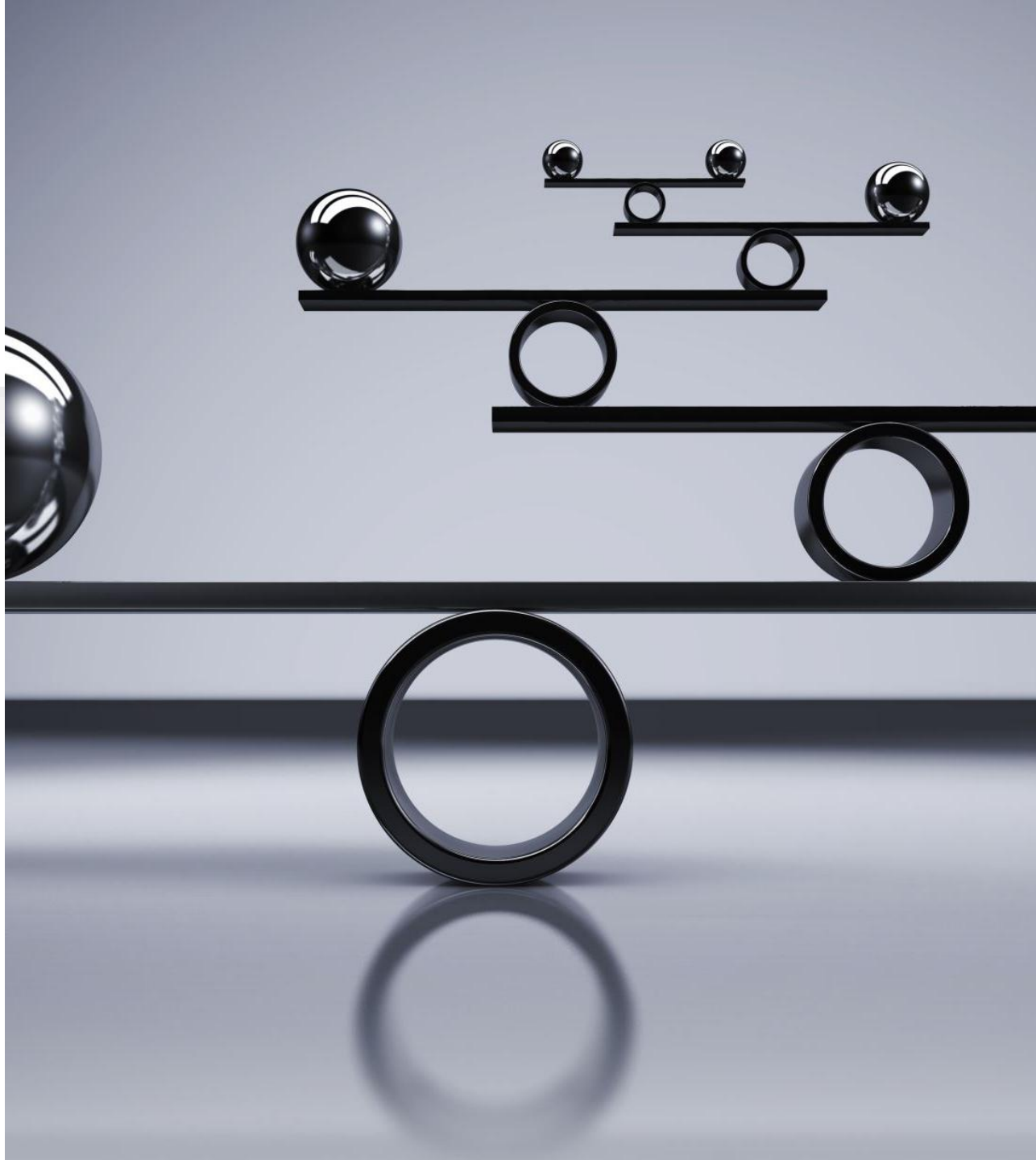
- **Maximum Flexibility:** Provides finest granularity of parameter selection, allowing removal of any individual weight regardless of position
- **Architecture Preservation:** Network structure remains completely intact; only individual parameter values are modified
- **High Compression Potential:** Can achieve very high sparsity levels by removing weights throughout the network without structural constraints
- **Storage Benefits:** Significant memory footprint reduction when using compressed sparse storage formats
- **Computational Challenges:** Sparse matrices require specialized execution kernels to realize computational benefits on most hardware
- **Irregular Memory Access:** Unpredictable sparsity patterns can degrade cache performance and memory bandwidth utilization

Magnitude-Based Weight Selection

The Standard Importance Criterion

Magnitude-based pruning assumes larger weights contribute more to network function:

- **Simple Heuristic:** Remove weights with smallest absolute values, keeping those with largest magnitudes
- **Threshold Selection:** Define cutoff value τ such that only weights with $|w| > \tau$ are retained
- **Percentile-Based:** Alternative approach removes fixed percentage of smallest weights regardless of absolute threshold
- **Layer-wise vs Global:** Can apply thresholds within each layer separately or globally across entire network
- **Computational Efficiency:** Very fast to compute; requires only single pass through parameters to rank by magnitude
- **Theoretical Justification:** Based on assumption that larger weights have greater influence on network outputs and gradients



Removing Entire Computational Units

Structured pruning eliminates entire neurons, channels, or layers, producing smaller dense models:

- **Hardware Compatibility:** Results in smaller dense models that execute efficiently on standard GPUs and CPUs without specialized sparse computation support
- **Computational Units:** Targets entire structures like neurons in dense layers, channels in convolutional layers, or complete layers
- **Immediate Benefits:** Provides direct computational savings by reducing FLOPs and memory bandwidth requirements
- **Architecture Changes:** Requires modification of network structure and connections between remaining components
- **Cache Efficiency:** Maintains regular memory access patterns that work well with hardware cache hierarchies
- **Framework Support:** Works seamlessly with existing deep learning frameworks and deployment pipelines

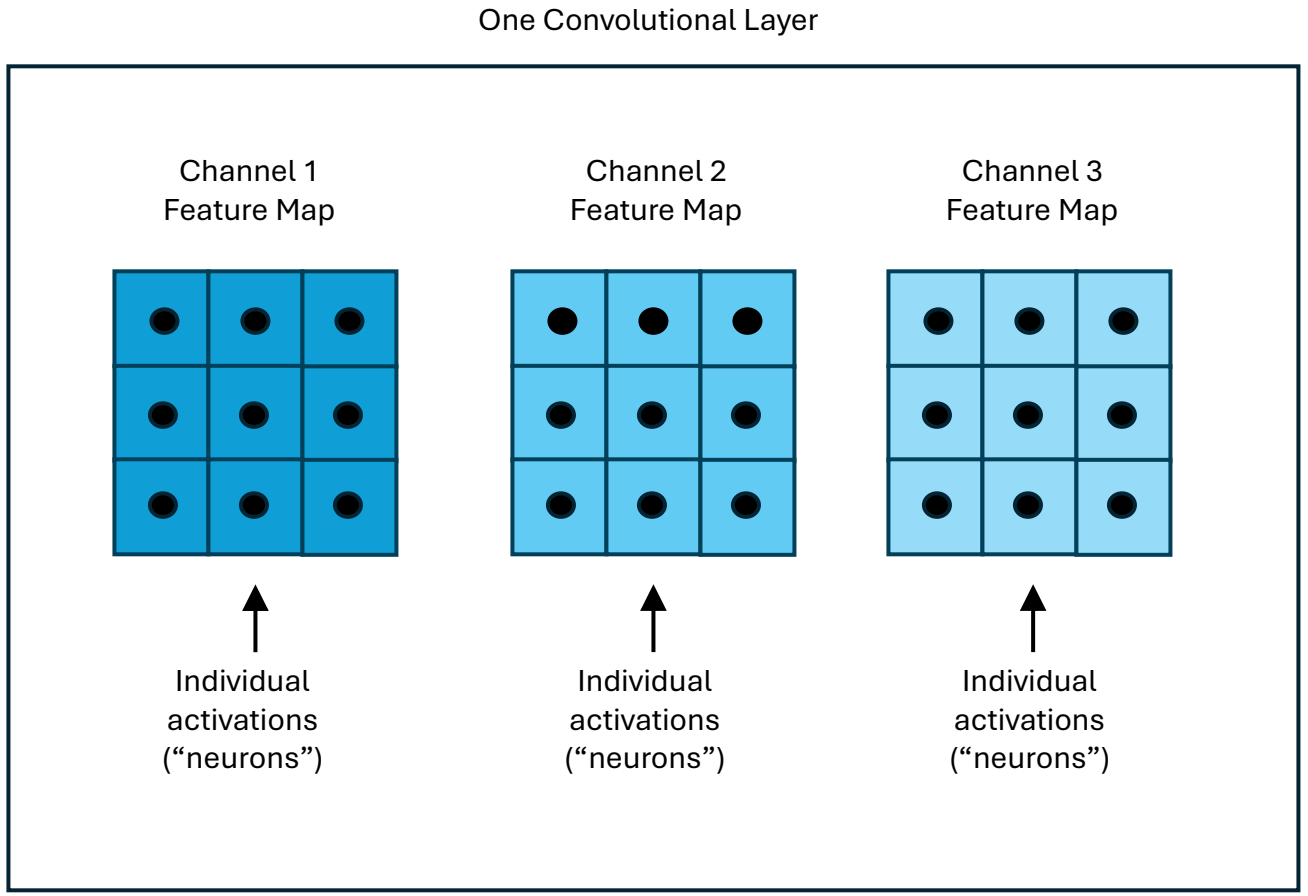
CNN Terminology: Layers, Channels, and Activations

$$Layer\ output\ shape = H \times W \times C$$

- **Layer:** the complete convolutional operation
- **Channel:** one output feature map
- **Neuron / activation:** one value at one location within a feature map

Connection to structured pruning

Pruning target	What is removed?
Neuron pruning	One individual activation/unit
Channel pruning	One entire feature map
Layer pruning	The entire convolutional layer



*A CNN activation tensor is commonly stored in either **NCHW** format—batch, channels, height, width—or **NHWC** format—batch, height, width, channels. Both describe the same layer output; they only differ in where the channel dimension is placed."*

Neuron Pruning in Dense Layers

Removes an entire neuron and its connections

$$\text{Layer width: } N \longrightarrow N - r$$

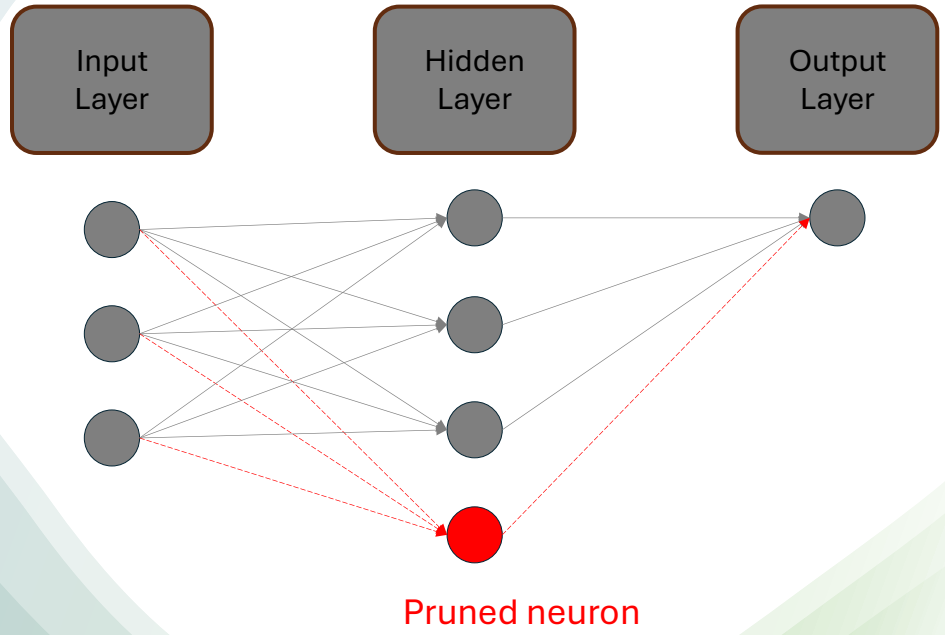
Pruning one neuron removes:

$$\text{Incoming weights} + \text{bias} + \text{Outgoing weights}$$

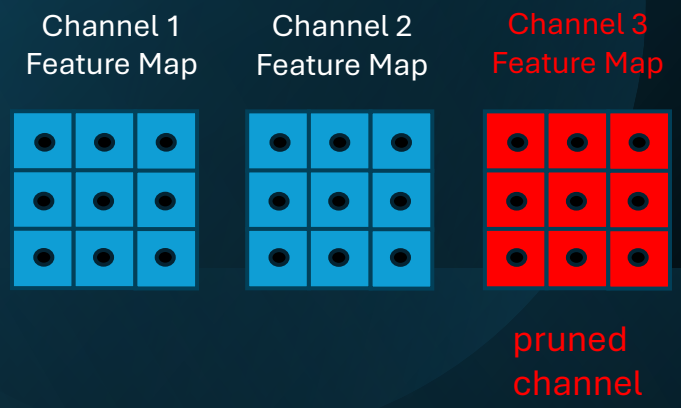
Results

- Smaller weight matrices
- Fewer activations to compute
- Lower parameter and memory cost

The layer remains dense — it just has fewer neurons.



Channel Pruning in CNNs



Removes an entire output feature map

$$\text{Output channels: } C_{\text{out}} \rightarrow C_{\text{out}} - r$$

When channel c is removed:

$$\text{Feature Map}_c \rightarrow removed$$

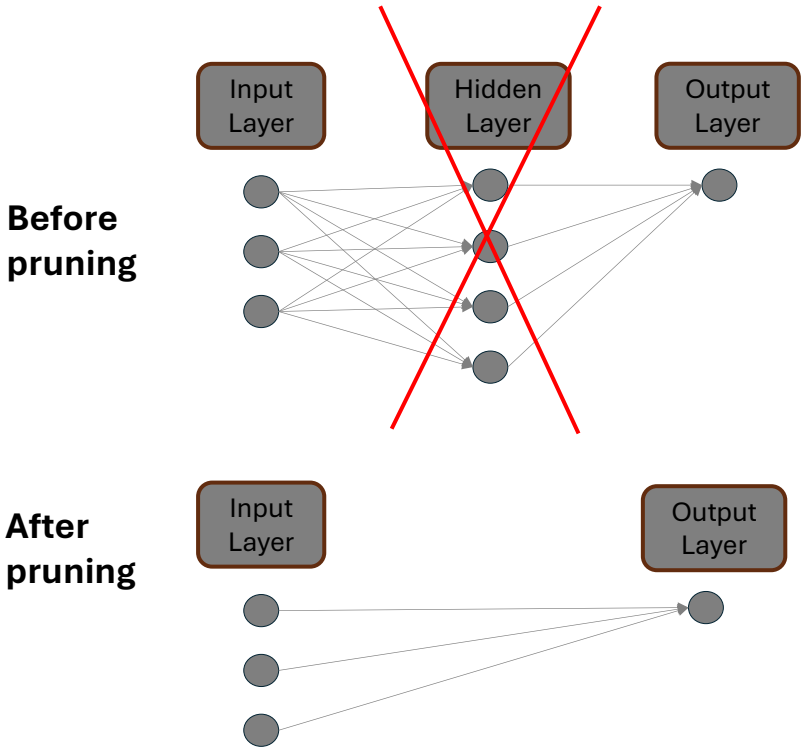
$$\text{Filter}_c \rightarrow \text{removed}$$

$$\text{Next layer input channel}_c \rightarrow \text{removed}$$

Results

- Fewer convolution operations
- Lower activation memory
- Smaller following layer

Structured pruning: removes complete channels, not individual weights.



Layer Pruning

Removes an entire layer or block of layers

$$\text{Depth: } D \longrightarrow D - r$$

$$\text{Layer}_{l-1} \longrightarrow \boxed{\text{Layer}_l} \longrightarrow \text{Layer}_{l+1}$$

$$\Downarrow$$

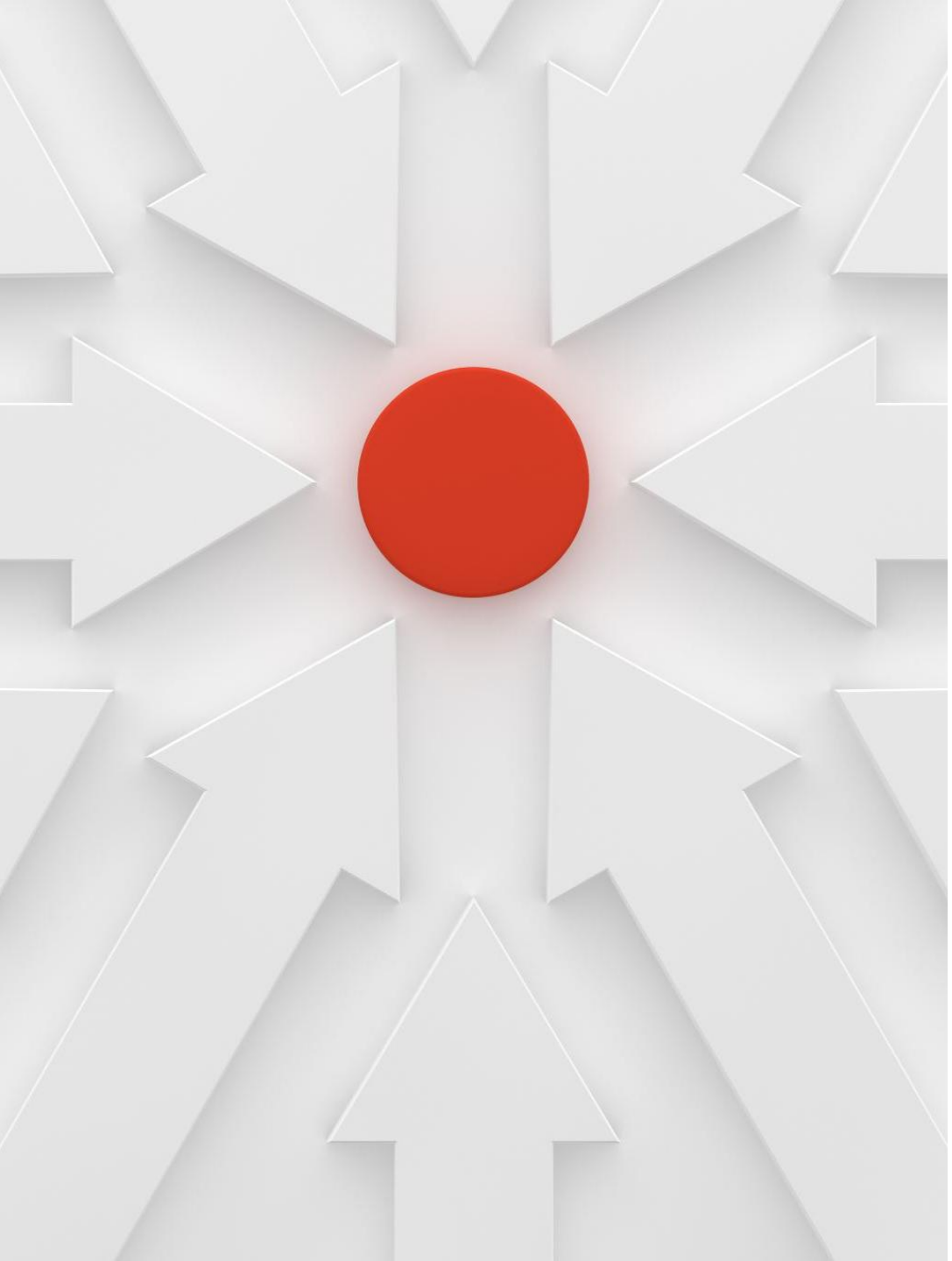
$$\text{Layer}_{l-1} \longrightarrow \text{Layer}_{l+1}$$

Requires compatible dimensions or adjustment of adjacent layers.

Results

- Large reduction in computation
- Fewer parameters and operations
- Highest risk of accuracy loss

Most aggressive form of structured pruning

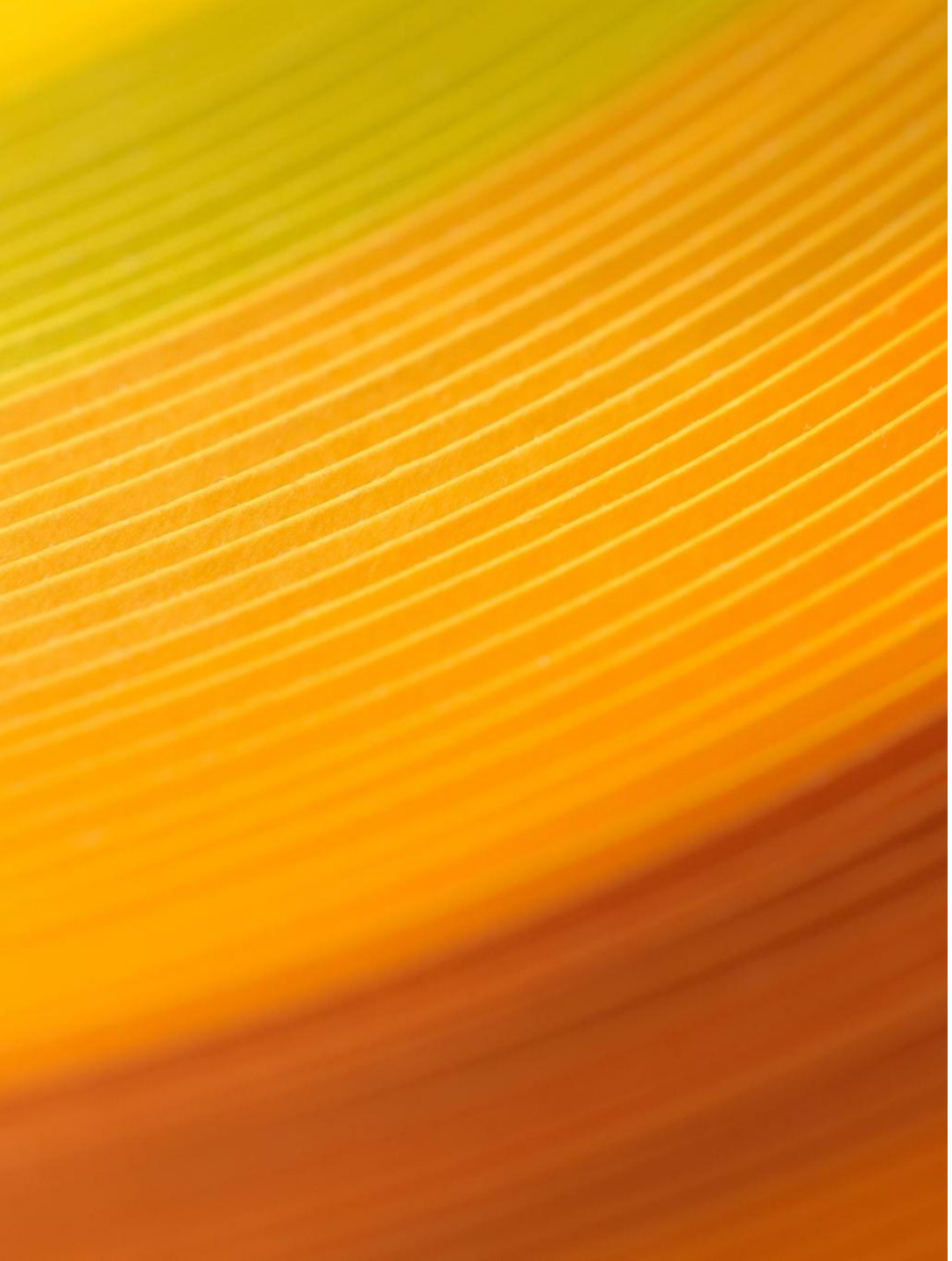


Target Structure Selection

Choosing What to Prune

Different pruning targets offer distinct advantages and challenges depending on the deployment context:

- **Neuron Pruning:** Most applicable to fully connected layers where individual neurons can be evaluated independently using activation magnitude, gradient information, or output contribution metrics
- **Channel/Filter Pruning:** Dominant approach for convolutional networks where channels with consistently low activation values suggest minimal feature contribution
- **Layer Pruning:** Most aggressive approach with highest efficiency gains but greatest accuracy risk, requiring analysis to ensure sufficient representational capacity
- **Selection Criteria:** Magnitude-based (larger weights/activations more important), gradient-based (training dynamics), activation-based (runtime behavior)
- **Hardware Alignment:** Target selection must match deployment platform capabilities to realize practical benefits rather than just theoretical improvements



Importance Metrics for Selection

Measuring Parameter Significance

Various metrics attempt to quantify the importance of different network structures:

- **Magnitude-Based Metrics:** Use weight magnitude, activation magnitude, or gradient magnitude as proxies for importance
- **Gradient-Based Importance:** Leverage first or second-order gradient information to estimate impact on loss function
- **Activation-Based Scoring:** Measure average activation values across representative dataset to identify consistently inactive units
- **Information-Theoretic Measures:** Use entropy, mutual information, or other information-theoretic quantities to assess contribution
- **Performance-Based Evaluation:** Temporarily remove structures and measure impact on validation accuracy or loss
- **Combination Approaches:** Integrate multiple metrics to create more robust importance estimates for pruning decisions

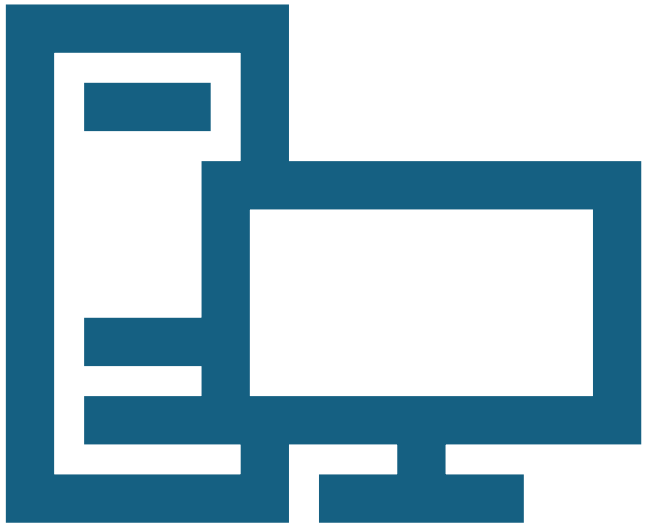


Hardware Compatibility Considerations

Matching Pruning to Deployment Targets

The effectiveness of different pruning approaches varies significantly across hardware platforms:

- **Standard CPUs and GPUs:** Optimized for dense matrix operations; unstructured pruning provides minimal computational benefit without specialized sparse libraries
- **Mobile Processors:** Include dedicated integer processing units and optimized inference engines; structured pruning works well with existing mobile ML frameworks
- **Microcontrollers:** Extremely limited memory and computational resources; structured pruning essential for fitting within flash storage constraints
- **Specialized Accelerators:** TPUs and ML accelerators may have specific sparsity support; some newer accelerators include sparse computation capabilities
- **Memory Hierarchy:** Target selection must consider cache efficiency, memory bandwidth, and access pattern regularity
- **Framework Ecosystem:** Pruning approach must align with available deployment frameworks and optimization tools



Standard Hardware Limitations

Dense Computation Optimization Reality

Most production hardware is optimized for dense rather than sparse computations:

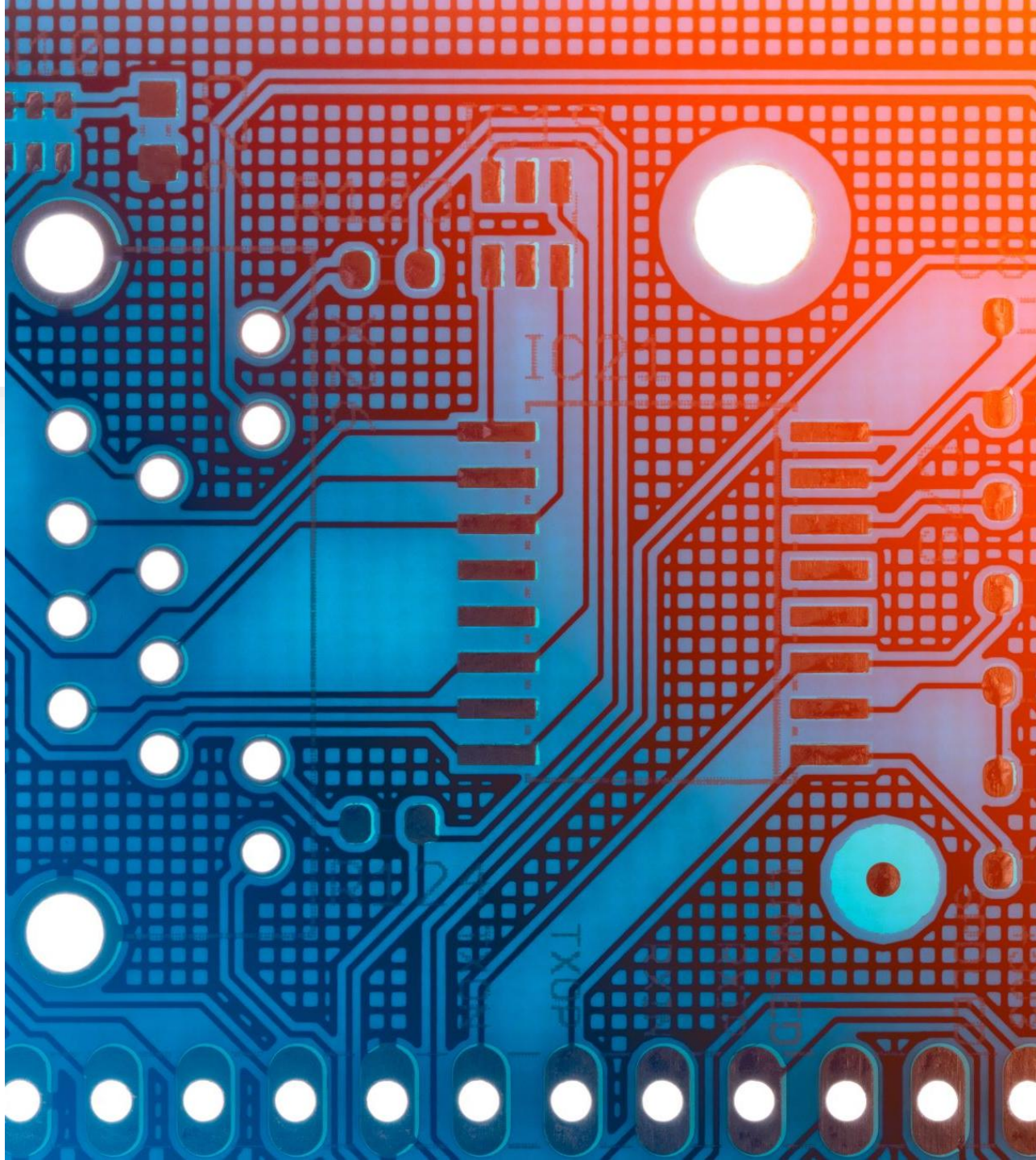
- **BLAS Libraries:** Basic Linear Algebra Subprograms optimized for dense matrix operations with predictable access patterns
- **GPU Architecture:** Designed for regular, coalesced memory access patterns that work poorly with irregular sparsity
- **CPU Cache Design:** Optimized for spatial and temporal locality that sparse matrices can disrupt with irregular access patterns
- **SIMD Instructions:** Vectorization requires regular data layouts that sparse representations often cannot provide
- **Memory Controllers:** Designed for burst transfers and prefetching that work best with contiguous dense data
- **Inference Engines:** TensorFlow Lite, ONNX Runtime, and similar frameworks primarily optimized for dense operations

Specialized Sparse Hardware

Emerging Sparse Computation Support

New hardware increasingly includes specialized support for sparse computations:

- **Sparse Matrix Units:** Some modern GPUs include dedicated hardware for sparse matrix operations
- **Custom Accelerators:** AI chips designed specifically for neural networks often include sparse computation capabilities
- **Structured Sparsity Support:** Hardware optimized for specific sparsity patterns like block-sparse or channel-wise sparsity
- **Software Libraries:** NVIDIA cuSPARSE, Intel MKL-DNN, and similar libraries provide optimized sparse kernels
- **Compiler Integration:** ML compilers increasingly include sparse optimization passes for supported hardware
- **Future Trends:** Industry moving toward better sparse computation support as sparsity becomes more prevalent



Memory Access Pattern Impact

Cache Performance and Sparsity

Memory access patterns significantly affect the practical performance of pruned models:

• **Cache Locality:** Dense matrices provide good spatial locality; sparse patterns can cause cache misses

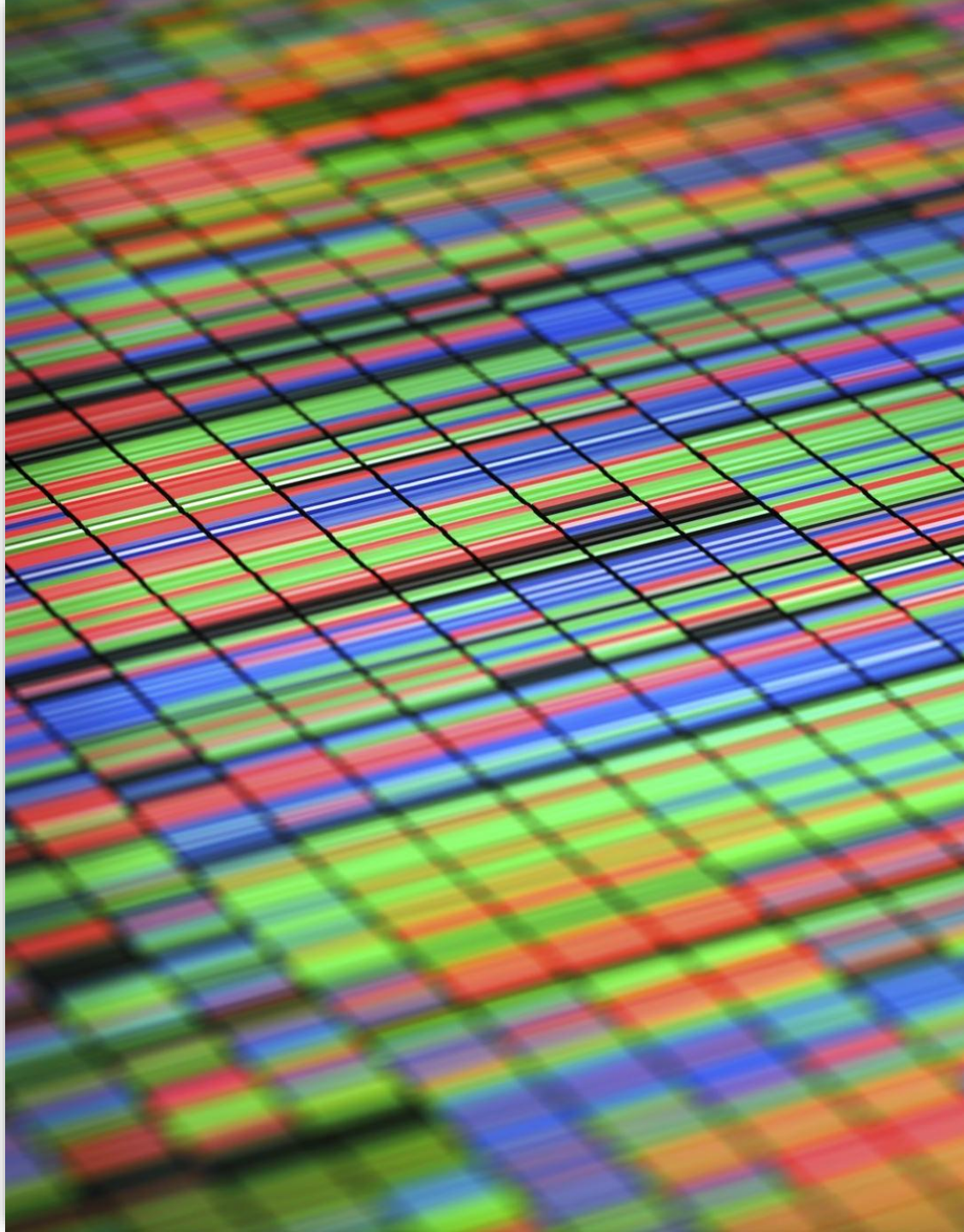
• **Prefetching Effectiveness:** Hardware prefetchers work well with predictable dense access patterns

• **Memory Bandwidth:** Irregular sparse access may not fully utilize available memory bandwidth

• **TLB Pressure:** Sparse patterns may access more memory pages, increasing translation lookaside buffer misses

• **Branch Prediction:** Sparse kernels often include conditionals that can cause branch mispredictions

• **Structured Benefits:** Structured pruning maintains regular access patterns that work well with cache hierarchies



Comparing Pruning Approaches

Aspect	Unstructured Pruning	Structured Pruning
Core Idea	Removes individual weights arbitrarily across the network.	Removes entire structural units (e.g., filters, channels).
Benefits	• Maximizes sparsity flexibility and compression. • Preserves the original network architecture.	• Works with standard hardware/libraries. • Provides immediate speedup (reduced FLOPs). • Better hardware performance and support.
Limitations	• Needs specialized software/hardware for speed. • Can have inefficient memory access. • May not speed up inference on standard hardware.	• Less flexible, can more easily hurt accuracy. • Generally achieves lower compression ratios.

Memory and Flash Constraints

Embedded System Realities

Most embedded systems have severe resource constraints that pruning must address:

- **Flash Storage Limits:** Most embedded systems have less than 1 MB of read-only storage, with many having only tens of kilobytes for programs and model data
- **SRAM Limitations:** Seldom more than 512 KB of static RAM available, often in low single digits for low-end devices
- **Power Constraints:** Operating in 1 mW-and-below power range usually prevents use of anything beyond SRAM due to energy consumption
- **Two Key Metrics:** Flash read-only storage availability and SRAM availability are the primary constraints for embedded ML deployment
- **Engineering Trade-offs:** Careful application design necessary to keep memory footprint low while maintaining acceptable performance

TensorFlow Lite for Microcontrollers Constraints

Framework Design for Resource Constraints

TensorFlow Lite for Microcontrollers designed for extreme resource limitations:

- **Minimum Requirements:** Designed to work with as little as 20 KB of flash and 4 KB of SRAM
- **No Dynamic Allocation:** Uses fixed memory arenas to avoid fragmentation and provide predictable memory usage
- **Operator Subset:** Supports subset of TensorFlow operations optimized for microcontroller deployment
- **Quantization Focus:** Emphasizes 8-bit quantized operations to reduce memory and computational requirements
- **Code Size Optimization:** Framework itself designed to have minimal code footprint through careful engineering
- **Platform Adaptation:** Provides specialization mechanisms for platform-specific optimizations while maintaining portability

Flash Usage Estimation

Calculating Read-Only Memory Requirements

Flash memory usage consists of several components that must be carefully budgeted:

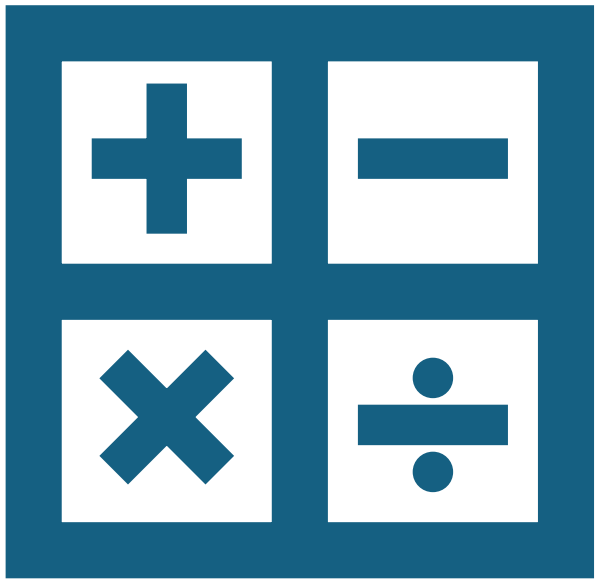
- **Operating System Size:** RTOS footprint varies based on enabled features; sample "hello world" program provides baseline
- **TensorFlow Lite Code:** ML framework program logic, operator implementations, and core arithmetic functions
- **Model Data Size:** Weights and biases stored as C data arrays; directly mapped into flash memory
- **Application Code:** Sensor access, preprocessing logic, postprocessing, and user interface components
- **External Libraries:** Additional dependencies like FFT libraries or communication stacks
- **USB/Wireless Stacks:** Communication modules can consume significant program space if needed

Model Size Calculation

Estimating Model Memory Requirements

Model size can be estimated by counting weights across all layers:

- **Fully Connected Layers:** Number of weights equals input vector size multiplied by output vector size
- **Convolutional Layers:** Filter width × height × input channels × number of filters, plus bias vectors
- **Quantization Impact:** 32-bit floating-point weights vs 8-bit quantized weights provide 4× size difference
- **TensorFlow Lite Export:** Export to TFLite file provides exact size figure as file will be directly mapped into flash
- **Keras Model Summary:** model.summary() method lists total number of trainable parameters
- **Compression Techniques:** Additional compression beyond quantization can further reduce model size



RAM Usage Components

Dynamic Memory Requirements

Modifiable memory usage varies during program execution and includes several components:

- **Operating System RAM:** RTOS memory requirements for different configuration options, including communication stack buffers
- **Framework Data Structures:** TensorFlow Lite interpreter, OpResolver, and runtime classes requiring few kilobytes
- **Model Memory Arena:** Activation tensors and intermediate results requiring careful planning and sizing
- **Application Memory:** Buffers for sensor data, preprocessing intermediates, and business logic
- **Stack Usage:** Function call overhead and local variables, particularly important for recursive or deep call chains
- **Global vs Local Objects:** Framework objects recommended as global/static to catch memory errors at link time rather than runtime

Model Memory Arena Planning

Managing Activation Tensor Memory

Neural network execution requires temporary storage for intermediate results:

- **Activation Lifetimes:** Results of one layer feed into subsequent operations and must be kept for varying durations
- **Memory Planning:** Framework calculates optimal memory layout to fit all temporary buffers into minimal arena size
- **Arena Sizing:** If arena too small, error occurs during model loading with message indicating required size
- **Conservative Approach:** Start with oversized arena, then reduce based on actual usage measurements
- **Layer Dependencies:** Graph structure determines which activations can share memory based on non-overlapping lifetimes
- **Memory Reuse:** Advanced planning allows reusing memory buffers for non-concurrent tensor operations

Quiz 1 – Question 1

Which of the following best describes the primary motivation for pruning neural networks?

A) Increase training speed
B) Remove redundant parameters that contribute minimally to predictions
C) Add more layers to improve accuracy
D) Convert models to different frameworks

Quiz 1 – Question 2

The mathematical formulation minimize L(W) subject to $||W||_0 < k$ is intractable because:

A) The loss function is non-convex

B) Hardware constraints prevent optimization

C) Neural networks are too complex

D) The ℓ_0-norm constraint is discrete and requires evaluating exponential combinations

Quiz 1 – Question 3

Unstructured pruning differs from structured pruning in that it:

A) Removes individual weights while preserving network architecture

B) Always provides better accuracy

C) Only works on convolutional layers

D) Requires specialized training procedures

Quiz 1 – Question 4

For microcontroller deployment, structured pruning is generally preferred because:

A) It always achieves higher compression ratios

B) It requires less computational resources to implement

C) It provides predictable memory access patterns and works with standard libraries

D) It preserves more accuracy than unstructured approaches

Quiz 1 – Question 5

The primary challenge with unstructured pruning on standard hardware is:

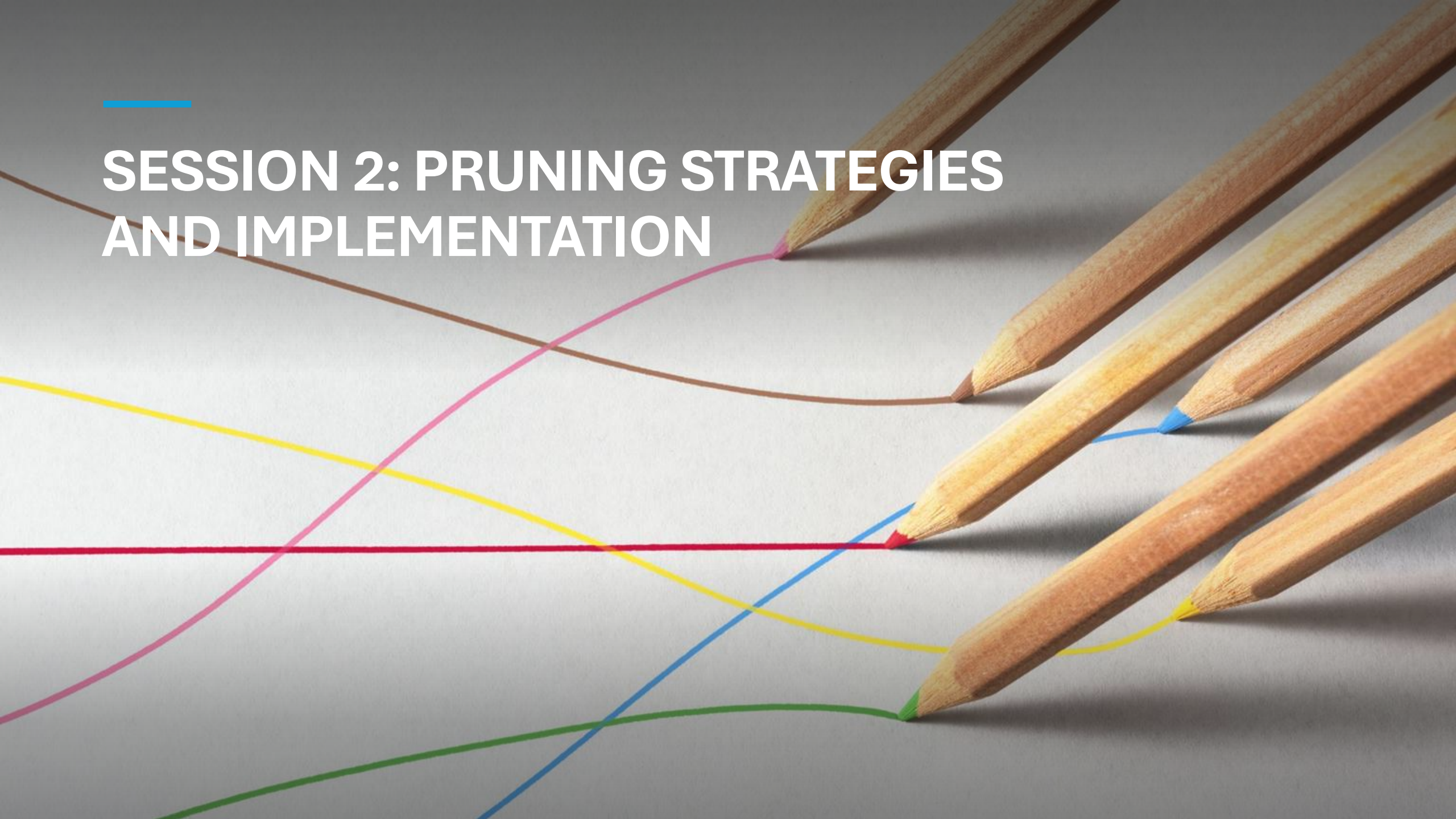
A) Higher memory usage

B) Increased model complexity

C) Sparse matrices require specialized execution support for computational benefits

D) Poor accuracy retention

SESSION 2: PRUNING STRATEGIES AND IMPLEMENTATION



Iterative vs One-Shot Pruning

Strategic Approaches to Parameter Removal

The timing and methodology of parameter removal significantly impacts the balance between final model accuracy and optimization efficiency.

Iterative Pruning Process:

1. Train model to convergence
2. Remove small percentage of least important parameters
3. Fine-tune to recover accuracy
4. Repeat until target sparsity achieved

One-Shot Pruning Process:

1. Train model to convergence
2. Remove target percentage of parameters in single step
3. Extensive fine-tuning to recover accuracy

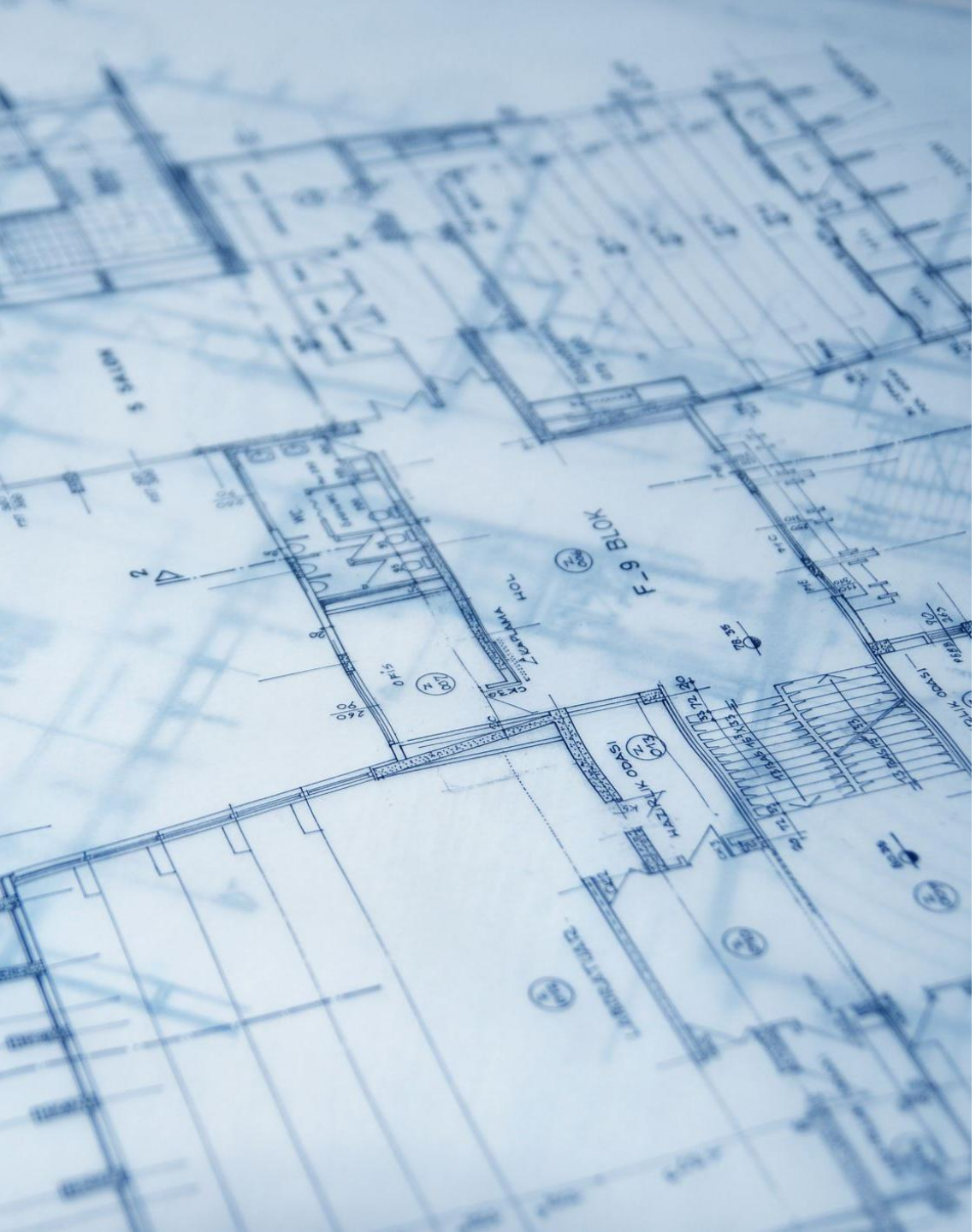
Trade-off Analysis:
Iterative pruning typically achieves better accuracy retention through gradual adaptation but requires multiple training cycles. One-shot pruning completes faster but risks greater accuracy degradation due to sudden structural changes.

Iterative Pruning: Gradual Approach

Progressive Parameter Removal

Iterative pruning implements a gradual approach to structure removal through multiple cycles:

- **Cycle Structure:** Each cycle removes small subset of structures based on importance metrics, followed by fine-tuning to adapt to modifications
- **Accuracy Preservation:** Distributing structural modifications across multiple iterations allows network to maintain performance while achieving efficiency
- **Example Results:** Removing 27% of channels through 3 iterations: accuracy drops from 0.995 to 0.971 initially, fine-tuning restores to 0.992, final accuracy 0.991 (0.4% reduction)
- **Resource Requirements:** More computationally intensive due to multiple fine-tuning cycles, but superior accuracy retention
- **Best Practices:** Smaller pruning percentages per iteration with thorough fine-tuning between cycles yield optimal results
- **Network Adaptation:** Allows remaining parameters to adjust gradually to compensate for removed structures



One-Shot Pruning: Aggressive Approach

One-shot pruning removes multiple architectural components simultaneously:

- **Aggressive Compression:** Eliminates all target parameters at once, causing substantial structural changes requiring comprehensive retraining
- **Accuracy Risk:** Removing 27% of channels simultaneously can drop accuracy from 0.995 to 0.914, recovering only to 0.943 (5% degradation vs 0.4% iterative)
- **Time Efficiency:** Completes optimization faster by avoiding multiple pruning-retraining cycles
- **Use Cases:** Appropriate when deployment speed prioritized over optimal accuracy retention
- **Implementation Consideration:** Requires careful selection of pruning percentage to balance efficiency with acceptable accuracy loss
- **Recovery Challenges:** Large structural changes difficult for network to compensate through fine-tuning alone



Pruning Strategy Selection Factors

Choosing the Right Approach

Several key factors influence the choice between iterative and one-shot pruning strategies:

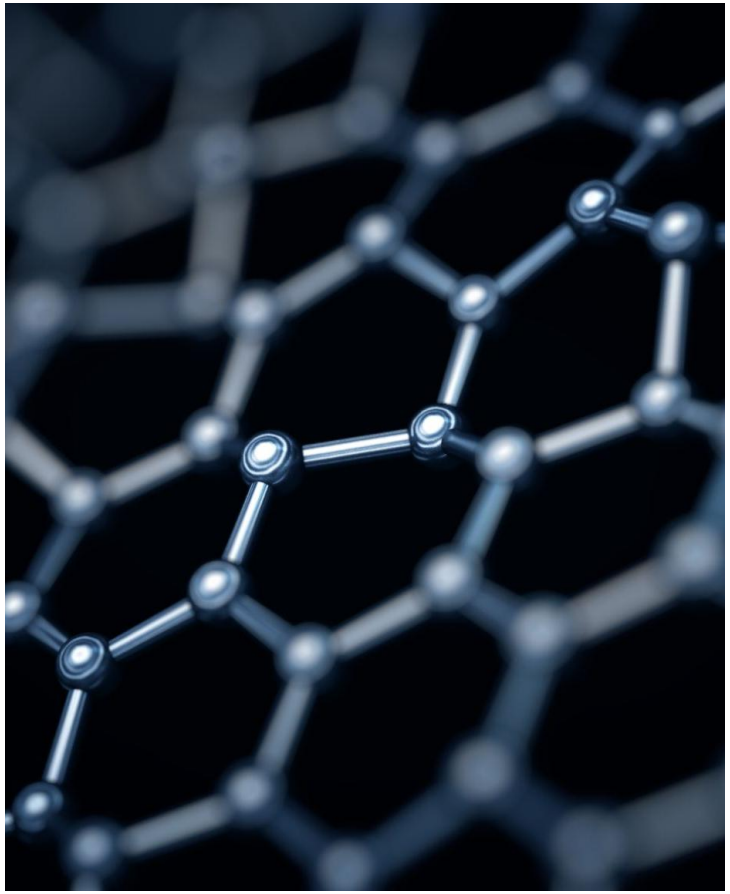
- **Sparsity Target:** Higher reduction targets often necessitate iterative approaches to maintain accuracy; moderate sparsity achievable through one-shot methods
- **Computational Resources:** Iterative pruning demands substantial resources for multiple fine-tuning cycles; one-shot approaches require fewer resources
- **Performance Requirements:** Applications with strict accuracy requirements benefit from gradual iterative pruning; flexible constraints may accommodate aggressive approaches
- **Development Timeline:** One-shot methods enable faster deployment when time limited; iterative approaches achieve superior results with sufficient optimization time
- **Hardware Constraints:** Target platform capabilities influence strategy selection based on memory and computational limitations
- **Model Characteristics:** Some architectures more resilient to aggressive pruning; others require gradual adaptation

Fine-Tuning Considerations

Recovery After Parameter Removal

Fine-tuning is critical for recovering accuracy after pruning operations:

- **Learning Rate Selection:** Lower learning rates often work better for fine-tuning pruned models to avoid destabilizing remaining parameters
- **Training Duration:** May require fewer epochs than original training since network structure already learned
- **Data Requirements:** Same training data used for fine-tuning; additional data can sometimes improve recovery
- **Regularization Adjustments:** May need to modify regularization strength since effective network capacity has been reduced
- **Early Stopping:** Monitor validation accuracy carefully to avoid overfitting with reduced model capacity
- **Multiple Rounds:** Iterative pruning requires balancing fine-tuning duration with total optimization time

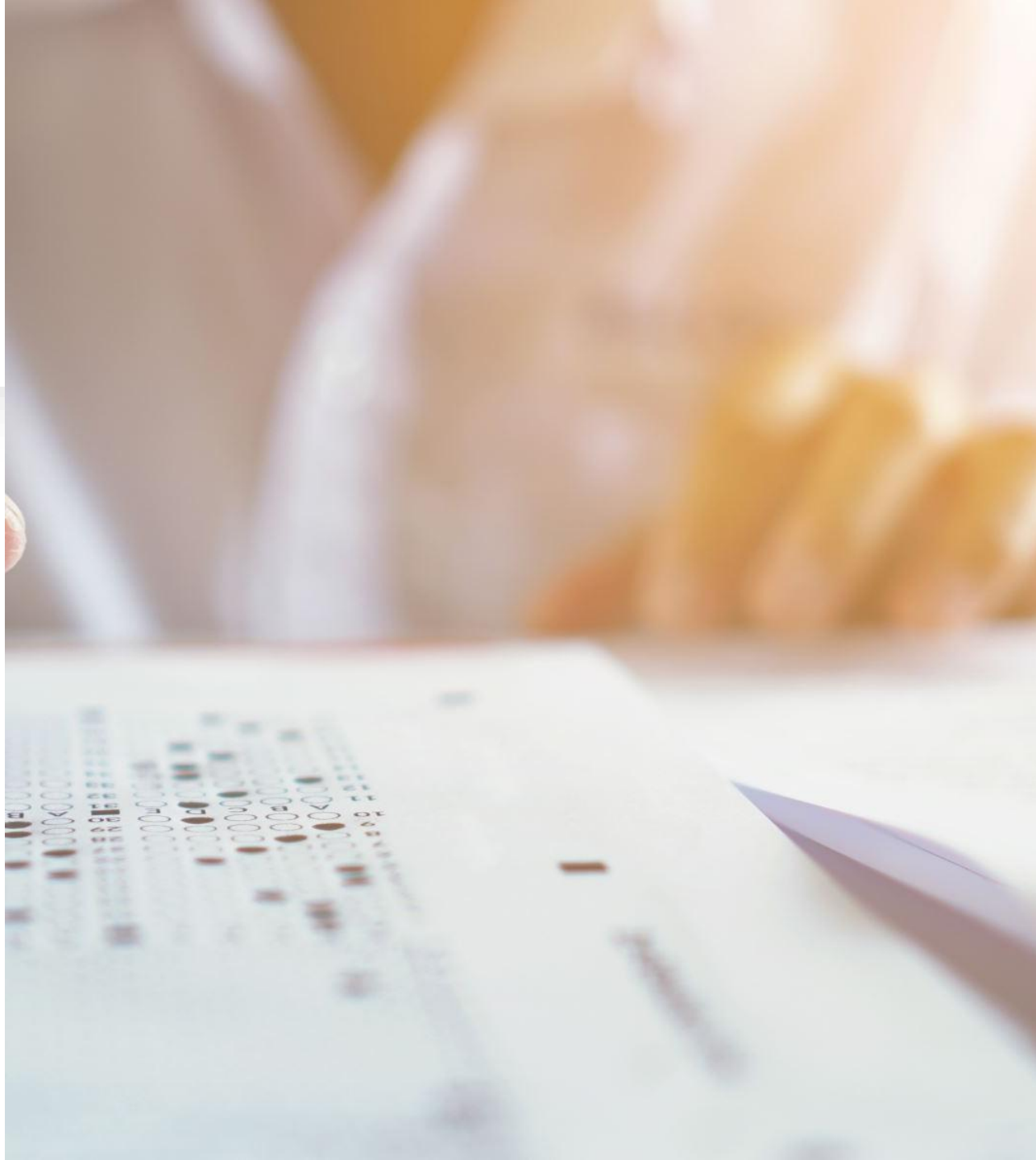


The Lottery Ticket Hypothesis

Discovering Winning Subnetworks

The Lottery Ticket Hypothesis challenges conventional pruning by suggesting large networks contain small, efficient subnetworks:

- **Key Insight:** Rather than viewing pruning as post-training compression, it may reveal inherently efficient subnetworks that could be trained directly
- **Winning Tickets:** Small, well-initialized subnetworks within larger models that achieve comparable accuracy when trained in isolation
- **Training Implications:** If winning tickets can be identified early, we might avoid overparameterization entirely
- **Current Limitations:** Finding winning tickets currently requires full training cycles, limiting practical benefits
- **Research Direction:** Ongoing work explores early detection methods to make lottery ticket approaches more practical
- **Theoretical Impact:** Suggests overparameterization may not be necessary for effective learning



Lottery Ticket Methodology

Iterative Discovery Process

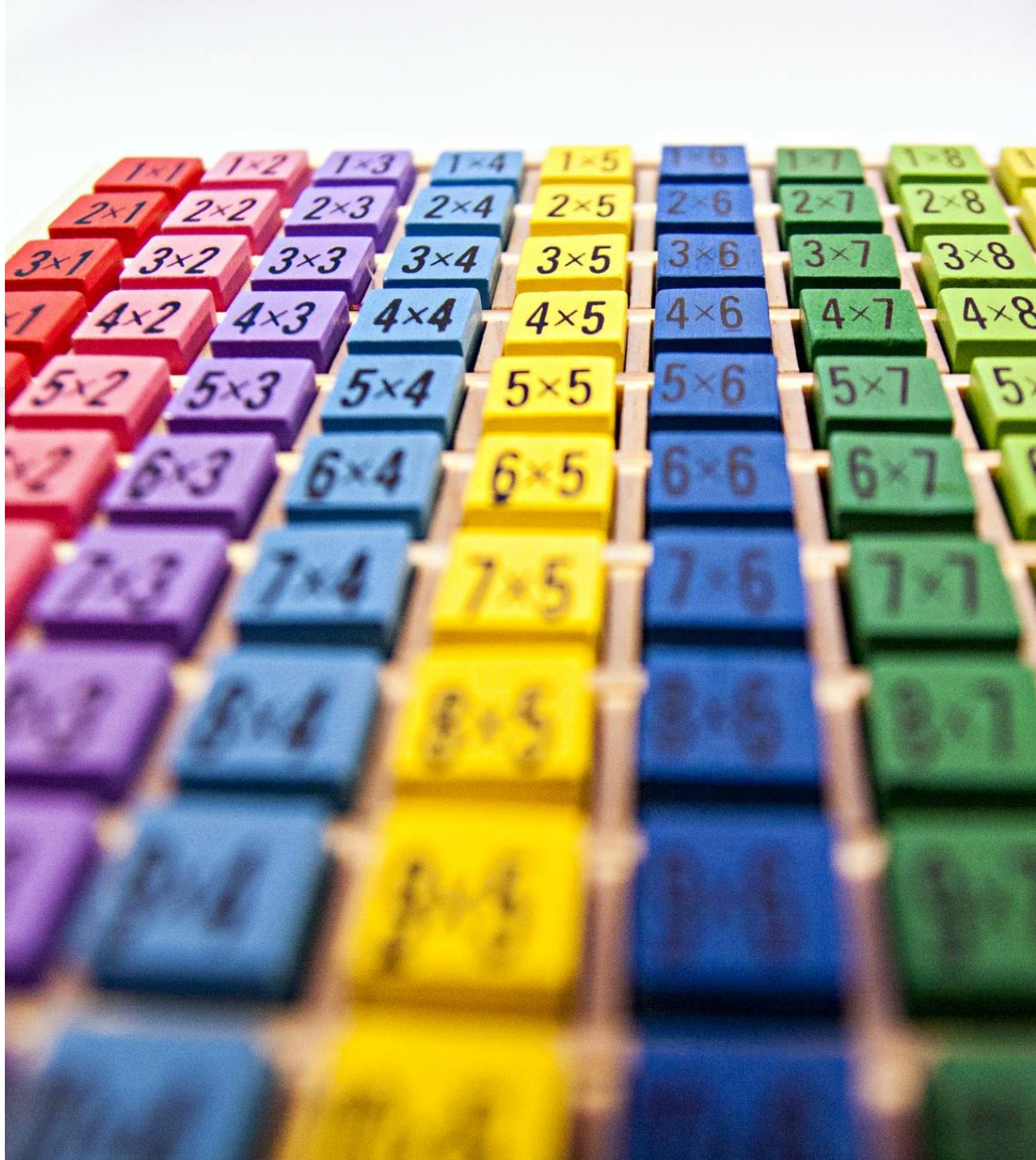
The Lottery Ticket Hypothesis validation follows a specific iterative process:

1. **Random Initialization:** Start with large network using random weight initialization
2. **Train to Convergence:** Train full network until optimal performance achieved
3. **Magnitude-Based Pruning:** Remove lowest-magnitude weights based on trained values
4. **Weight Reset:** Reset remaining weights to original initialization values (critical step)
5. **Iteration:** Repeat process with pruned network structure
6. **Validation:** Verify final subnetwork achieves comparable or better accuracy

• **Importance of Initialization:** Winning tickets only retain performance when reset to original values, emphasizing critical role of initialization

• **Iterative Refinement:** Gradual pruning allows better adaptation than aggressive removal

• **Research Implications:** Suggests optimal subnetworks could potentially be identified and trained directly



Winning Ticket Properties

Characteristics of Efficient Subnetworks

Winning tickets exhibit specific properties that distinguish them from random sparse networks:

- **Initialization Dependence:** Performance tied to specific weight initialization values; random reinitialization destroys capability
- **Sparsity Tolerance:** Can maintain accuracy at sparsity levels that would destroy randomly pruned networks
- **Training Efficiency:** Often train faster than original dense network despite having learned sparse structure
- **Generalization:** Some winning tickets generalize across different datasets within same domain
- **Transferability:** Limited evidence for transfer between different architectures or problem domains
- **Scale Dependence:** Lottery ticket phenomenon more pronounced in larger networks with greater overparameterization

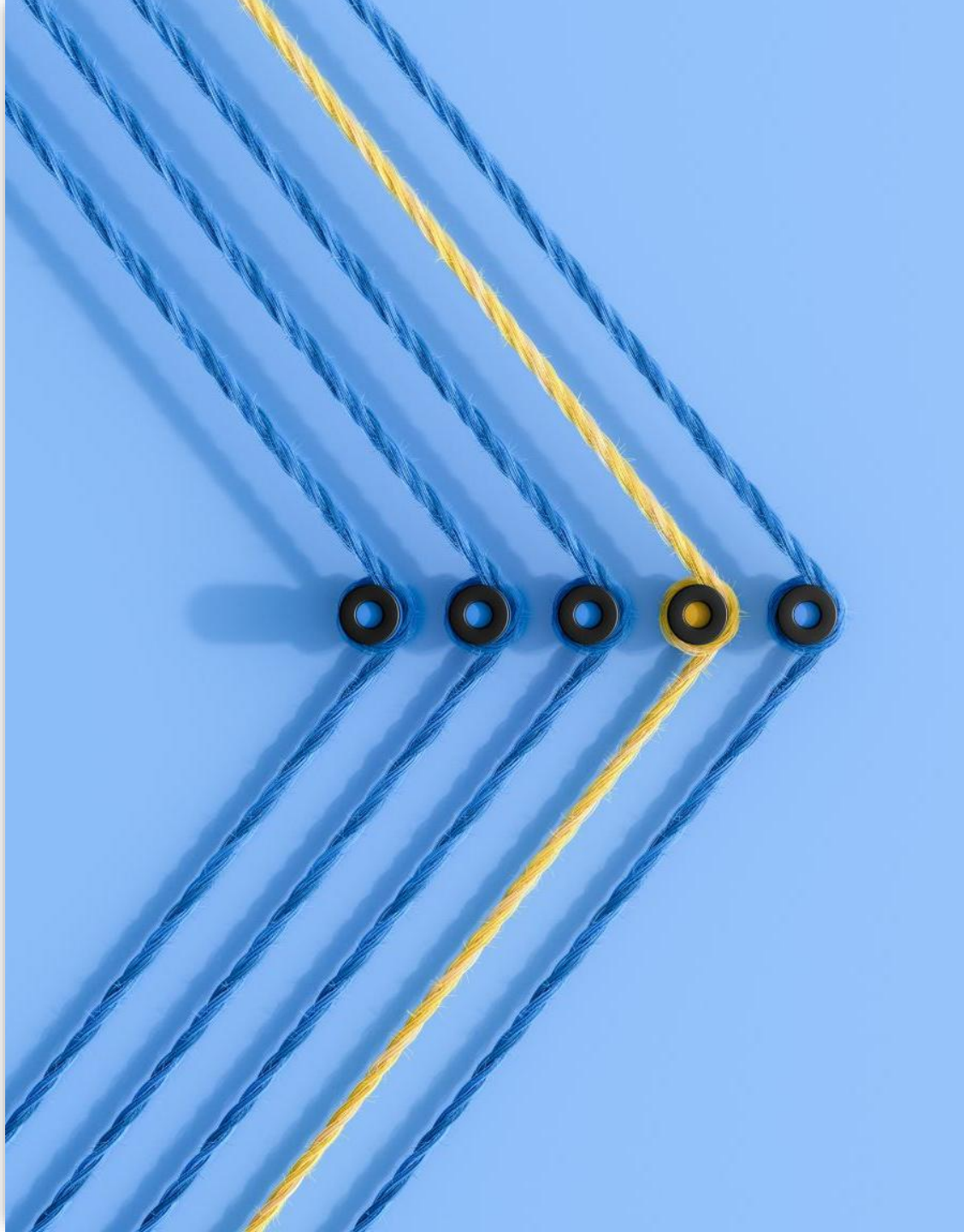
Framework Implementation

Practical Pruning with Modern Tools

PyTorch Implementation:
• torch.nn.utils.prune: Built-in magnitude and structured pruning capabilities
• Supports both structured and unstructured pruning methods
• Integration with standard training loops and model saving/loading

TensorFlow Approach:
• Model Optimization Toolkit for training-time pruning
• Applies sparsity during training rather than post-hoc
• Enables gradual sparsity introduction with automatic accuracy recovery

Framework Differences:
• PyTorch emphasizes post-training pruning with fine-tuning workflows
• TensorFlow promotes training-time sparsity for seamless integration
• Both support export to optimized deployment formats



PyTorch Pruning Utilities

Built-in Pruning Capabilities

PyTorch provides comprehensive pruning support through torch.nn.utils.prune module:

- **Magnitude Pruning:** l1_unstructured() and l1_structured() for removing weights based on magnitude
- **Random Pruning:** random_unstructured() and random_structured() for baseline comparison studies
- **Custom Pruning:** Framework for implementing custom pruning criteria beyond magnitude-based approaches
- **Global Pruning:** Capability to prune across entire model rather than layer-by-layer
- **Permanent Removal:** prune.remove() makes pruning permanent by actually deleting zero weights
- **Mask Management:** Automatic handling of pruning masks during forward and backward passes

TensorFlow Model Optimization Toolkit

Training-Time Sparsity Integration

TensorFlow's approach integrates pruning into the training process:

- **Sparsity Schedules:** Define how sparsity increases during training (constant, polynomial, exponential decay)
- **Training Integration:** Pruning applied during training allows model to adapt continuously to increasing sparsity
- **Automatic Recovery:** Model learns to compensate for removed parameters during training process
- **Deployment Optimization:** Seamless conversion to TensorFlow Lite with sparsity patterns preserved
- **Quantization Compatibility:** Works well with quantization for combined optimization benefits
- **Hardware Awareness:** Can target specific sparsity patterns optimized for particular hardware platforms

Model Choice for Optimization

Starting with Existing Architectures

For optimizing model and binary size, starting with existing models is highly recommended:

- **Focus on Data First:** Most fruitful investment is data gathering and improvement rather than architecture tweaking
- **Operational Support:** Existing models more likely to have supported and optimized operations on target devices
- **Proven Starting Points:** Available code samples cover different sensor inputs and provide good application starting points
- **Systematic Search:** Search for suitable architectures online before building custom solutions from scratch
- **Training Environment:** Custom architectures can be built in TensorFlow but porting to microcontrollers can be complex
- **Machine Learning Evolution:** Embedded ML software still evolving; existing models reduce risk of operational gaps

Ballpark Performance Figures

Accuracy and Size Trade-offs for Different Problems

Understanding current state-of-the-art helps plan achievable results:

Speech Wake-Word Model:
• Size: 18 KB model using 400,000 arithmetic operations
• Accuracy: 85% top-one accuracy distinguishing silence, unknown words, "yes," and "no"
• Practical Use: First-stage gatekeeper in larger cascade; errors tolerated by more complex downstream models
• Commercial Scale: 300-400 KB models with tens of millions of operations needed for acceptable wake-word accuracy

Performance Context:
• Training evaluation metric based on one-second clips for one-shot classification
• Streaming audio applications typically have lower practical accuracy due to temporal processing
• Commercial-quality datasets still needed for production-quality wake-word detection



Accelerometer Predictive Maintenance

Sensor Data Processing Capabilities

Predictive maintenance represents simpler ML applications with good performance characteristics:

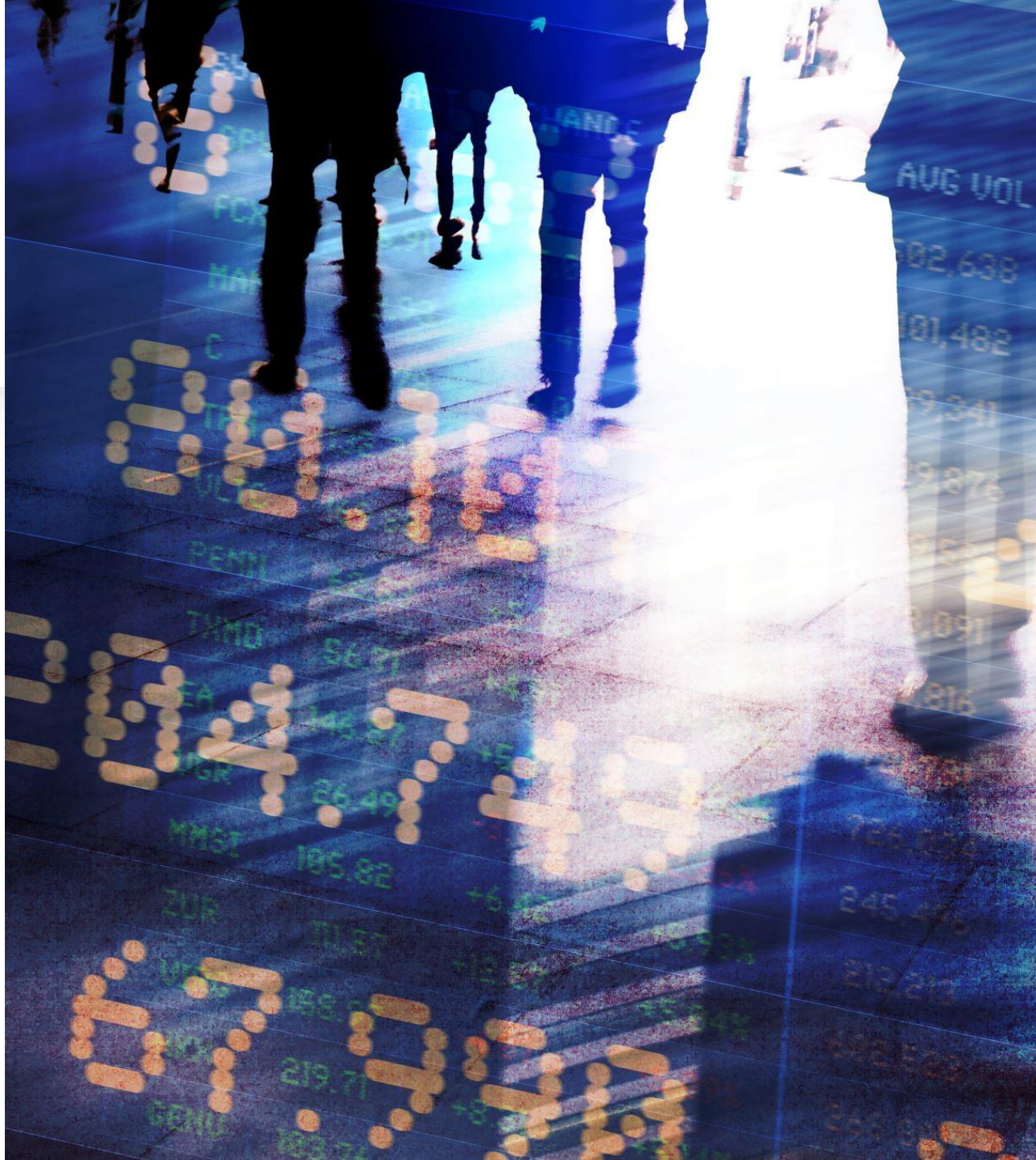
- **Model Size:** Few thousand weights, less than 10 KB total size, few hundred thousand arithmetic operations
- **Application Domain:** Detecting bearing failure patterns in motor accelerometer data through distinctive shaking signatures
- **Accuracy Expectations:** Better than 95% classification accuracy achievable for bearing failure detection
- **Complexity Scaling:** Model size and operations scale with problem difficulty (multiple moving parts, traveling machines)
- **Data Requirements:** Simpler than speech or vision; accelerometer patterns often more consistent and predictable
- **Deployment Advantages:** Small models fit easily within microcontroller constraints while maintaining high accuracy

Vision Model Performance

Person Presence Detection Capabilities

Computer vision on embedded platforms represents emerging capabilities with specific performance trade-offs:

- **Visual Wake Word Challenge:** Formal baseline for person detection requirements provides realistic performance expectations
- **Performance Metrics:** 90% accuracy with binary classification of 96×96 pixel monochrome images using 250 KB model and 60 million operations
- **Architecture Basis:** Results from scaled-down MobileNet v2 optimized for microcontroller memory constraints
- **ImageNet Comparison:** Similar-sized model (~500 KB) would achieve ~50% top-one accuracy on ImageNet-1,000 categories
- **Hardware Limitations:** Fully connected layer for 1,000 classes requires 100+ KB just for final layer parameters
- **Research Trajectory:** Accuracy expected to improve as more researchers tackle specialized embedded vision requirements

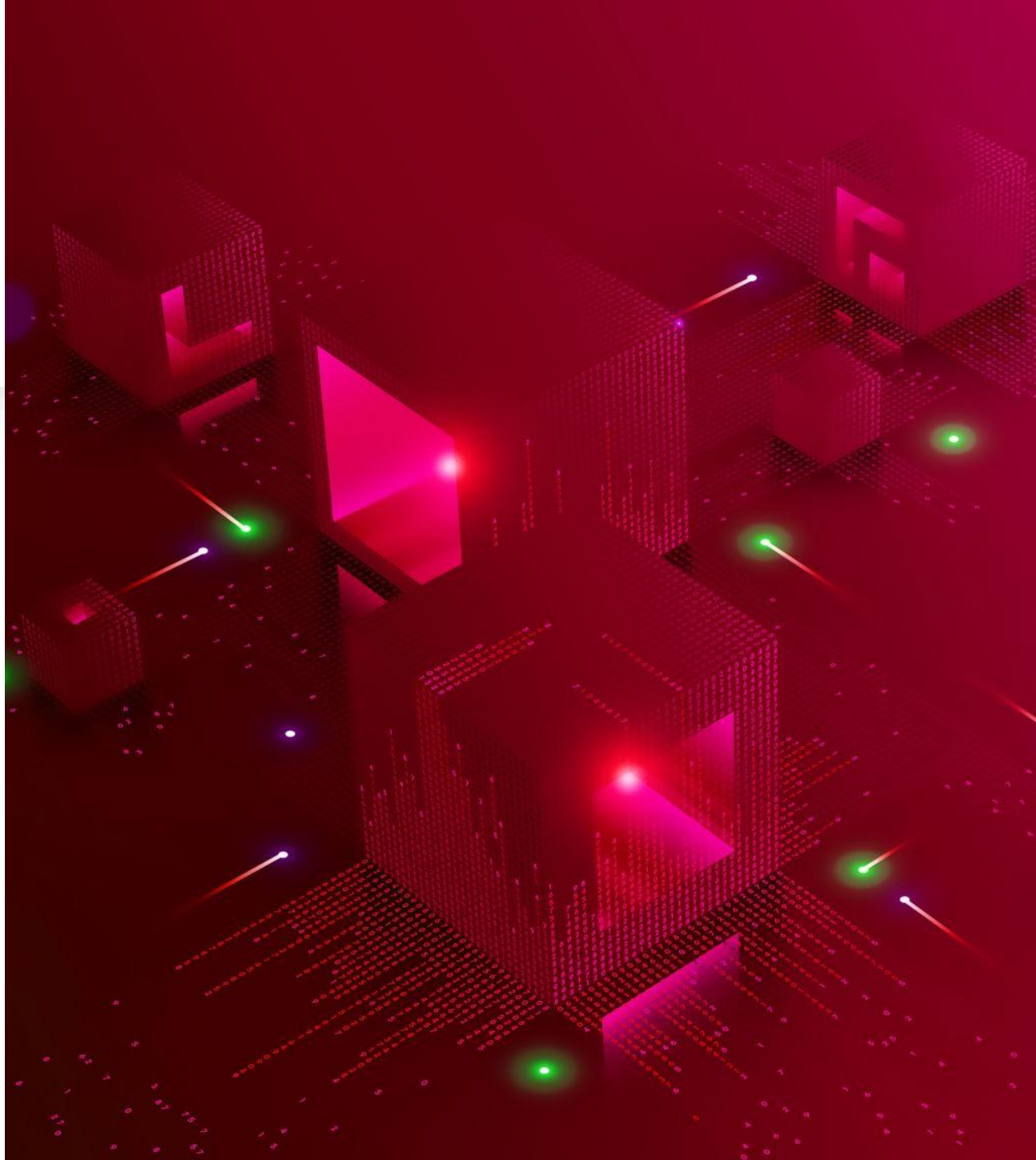


Model Architecture Considerations

Design Choices for Embedded Deployment

Architecture selection significantly impacts the success of pruning optimization:

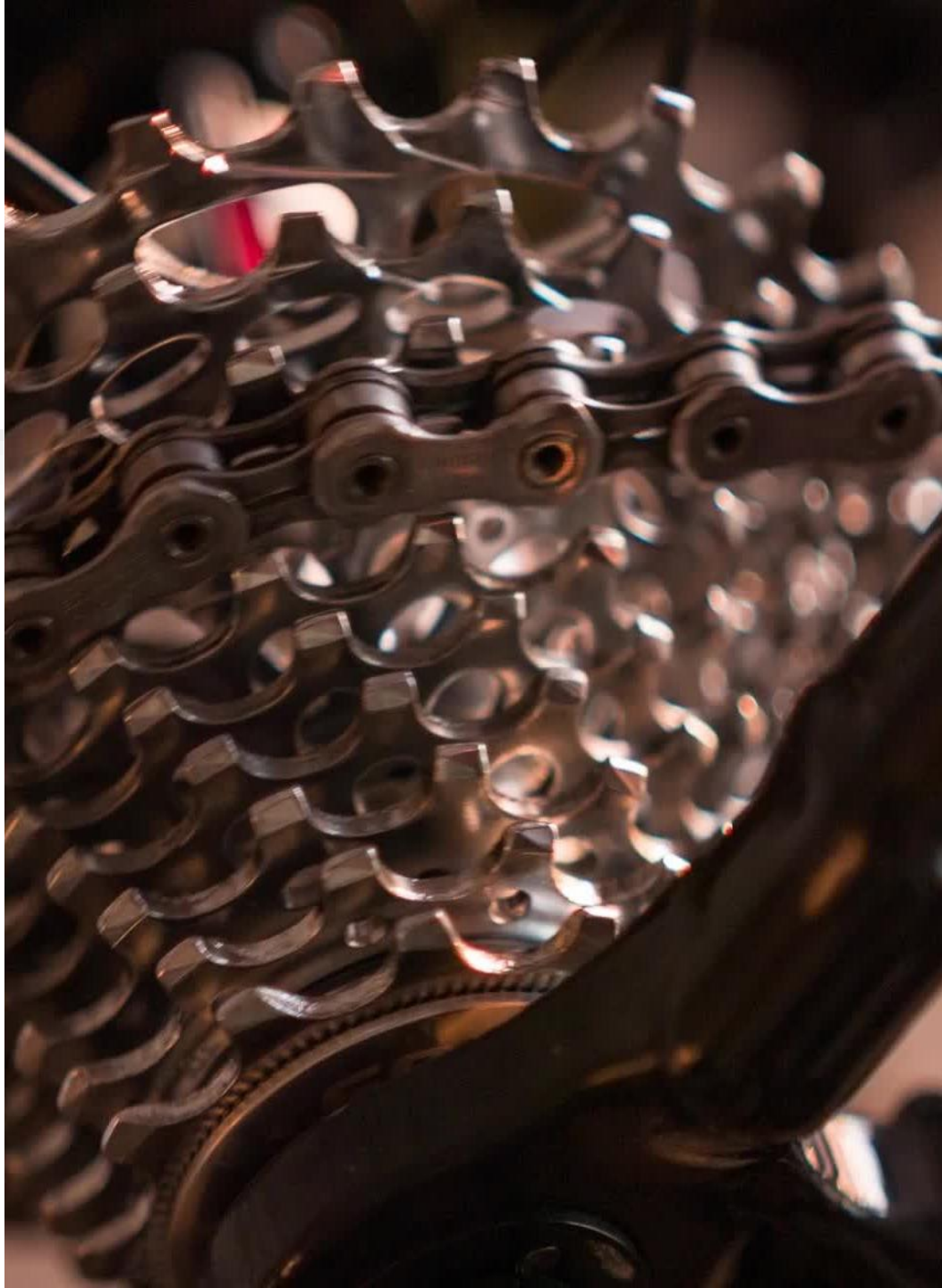
- **Layer Types:** Some layer types prune more gracefully than others; convolutions often more resilient than fully connected
- **Network Depth:** Deeper networks may have more redundancy but removing layers requires careful analysis
- **Width vs Depth:** Wider shallow networks vs narrow deep networks respond differently to structured pruning
- **Skip Connections:** Residual connections can help maintain information flow after aggressive pruning
- **Bottleneck Structures:** Some architectures naturally concentrate important parameters in specific layers
- **Activation Functions:** Choice of activation can affect which neurons are considered unimportant during pruning



Quiz 2 - Question 1

Iterative pruning typically achieves better accuracy retention than one-shot pruning because:

A) It uses different importance metrics
B) It allows gradual network adaptation through multiple fine-tuning cycles
C) It removes more parameters overall
D) It only works with structured pruning



Quiz 2 - Question 2

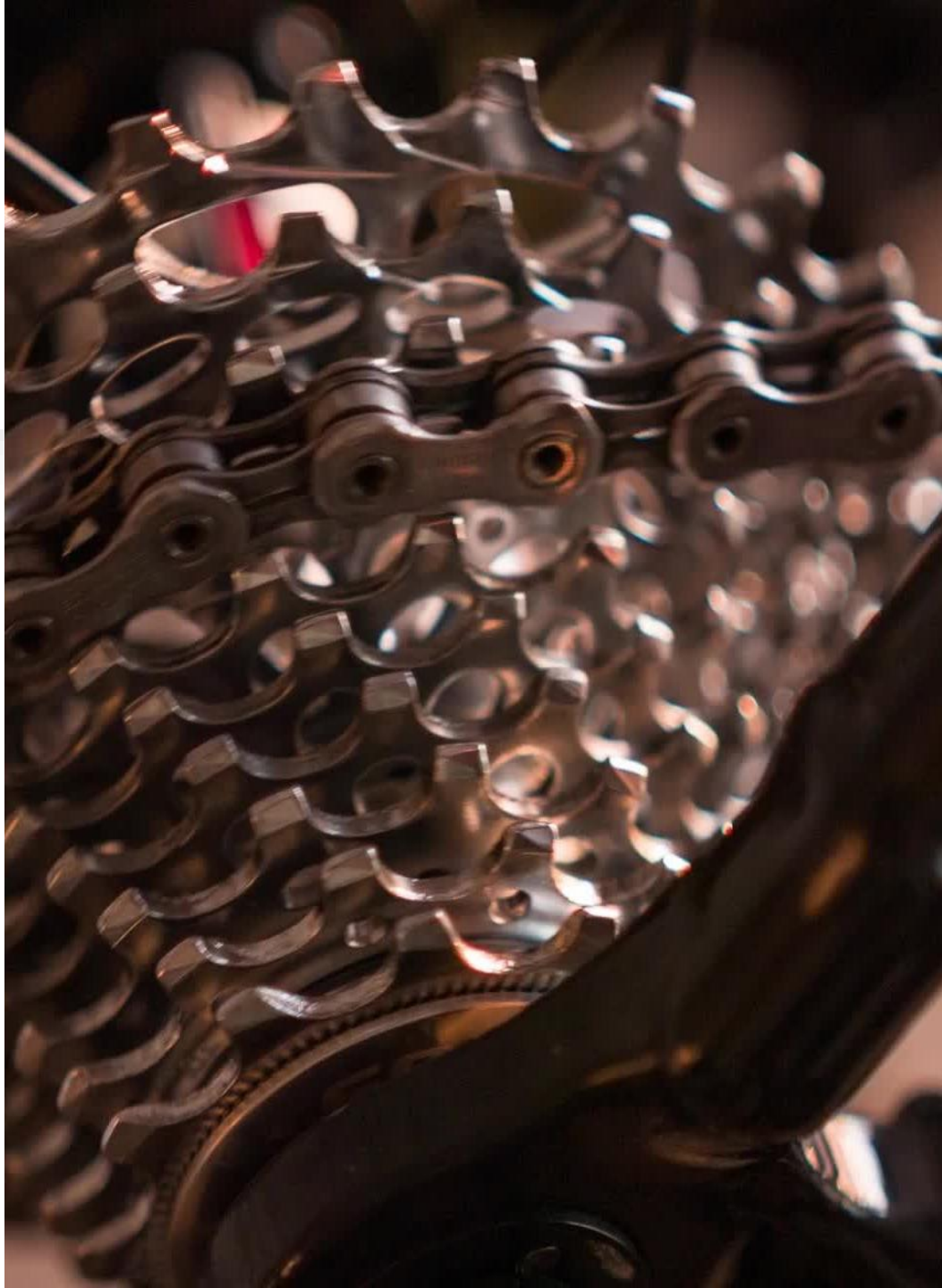
The Lottery Ticket Hypothesis suggests that:

A) Large networks contain small, well-initialized subnetworks that can achieve comparable accuracy

B) Pruning is unnecessary for neural networks

C) Random initialization is optimal for all networks

D) Only magnitude-based pruning is effective



Quiz 2 - Question 3

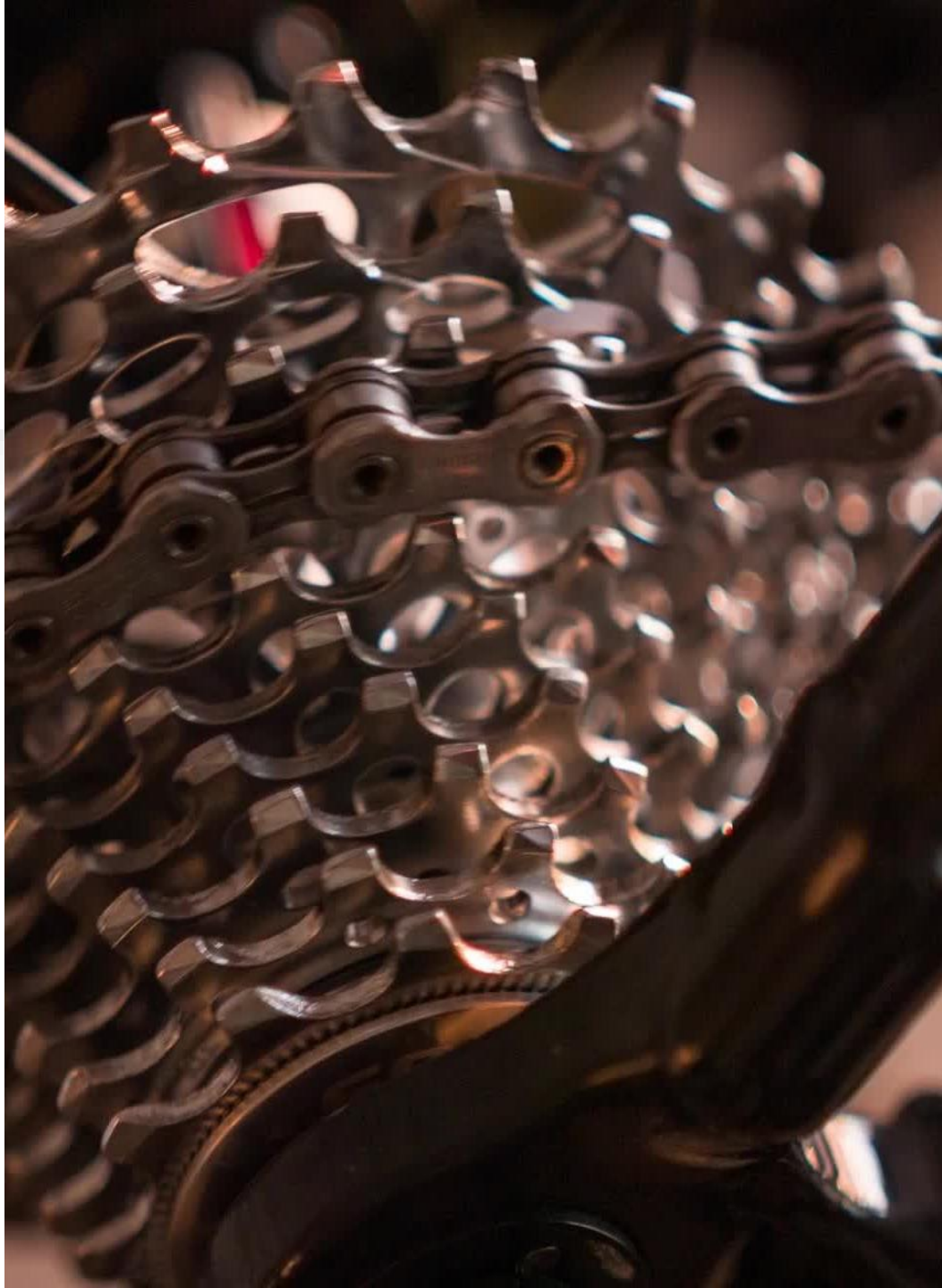
In the Lottery Ticket Hypothesis validation process, remaining weights are reset to:

A) Random values

B) Zero values

C) Their original initialization values

D) The average of all weights



Quiz 2 - Question 4

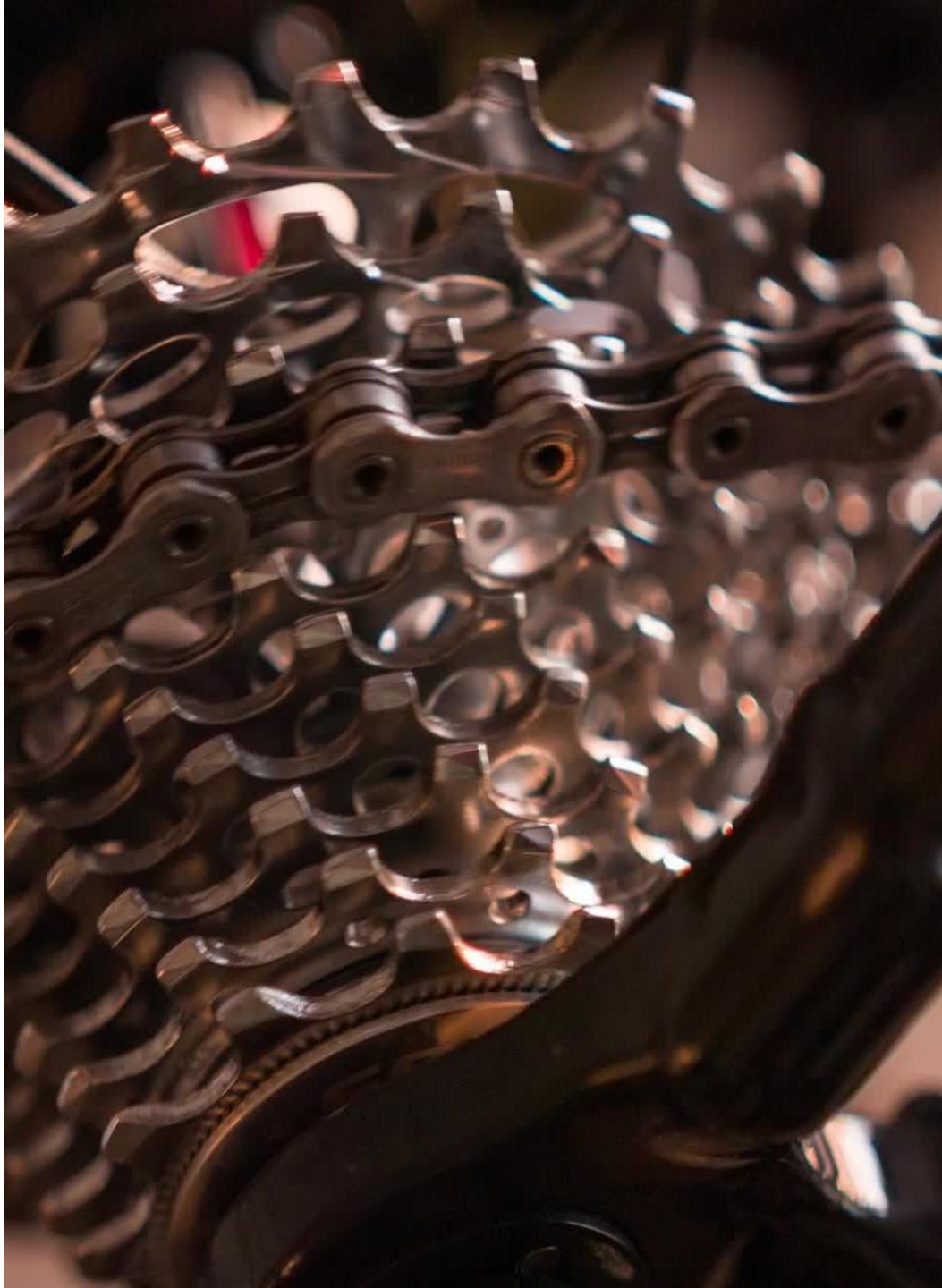
For resource-constrained deployment, it's recommended to start with existing models because:

A) They are always smaller

B) Operations are more likely to be supported and optimized on target devices

C) Custom models are illegal

D) They require less memory



Quiz 2 - Question 5

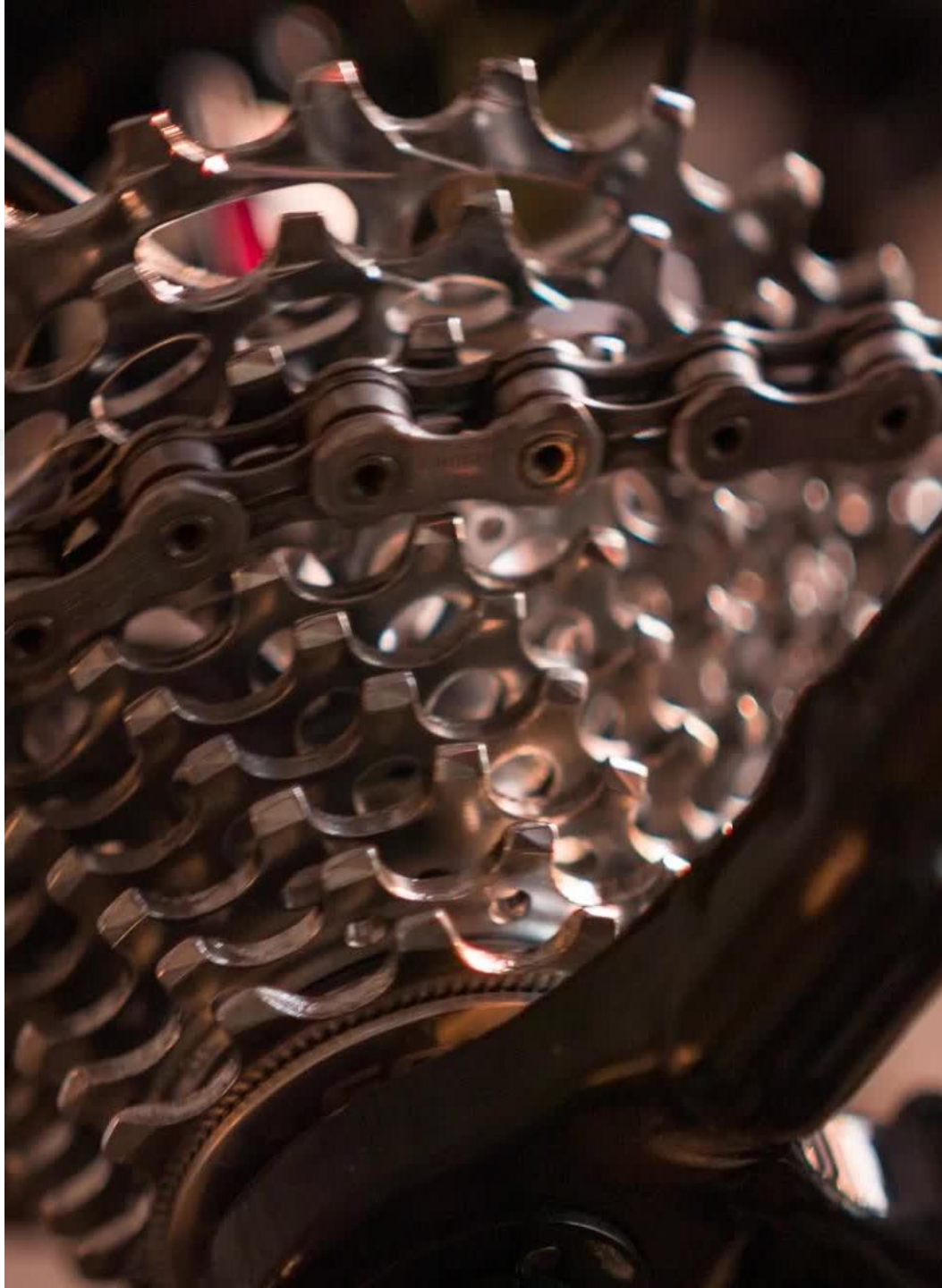
A speech wake-word model of 18 KB typically achieves:

A) 95% accuracy on complex speech recognition

B) 99% accuracy on all voice commands

C) 50% accuracy on all speech tasks

D) 85% accuracy distinguishing between silence, unknown words, "yes," and "no"



SESSION 3: ADVANCED TECHNIQUES AND DEPLOYMENT

Dynamic Pruning for Adaptive Systems

Runtime Optimization Strategies

Dynamic pruning adapts computational load based on input characteristics and system conditions:

- **Input-Dependent Sparsity:** Complex inputs receive full model capacity while simple inputs use heavily pruned versions
- **System-Aware Pruning:** Adapts sparsity based on available computational resources, battery level, or thermal constraints
- **Progressive Refinement:** Initial inference uses highly pruned model for rapid response, followed by full refinement if needed
- **Implementation Challenges:** Runtime overhead for pruning decisions, memory management for multiple model variants
- **Trade-offs:** Dynamic approaches exchange implementation complexity for improved user experience and resource efficiency
- **Use Cases:** Particularly valuable for mobile and embedded deployments with varying operating conditions

Activation-Conditioned Pruning

Input-Dependent Computational Adaptation

Activation-conditioned pruning selectively deactivates neurons or channels based on runtime patterns:

- **Dynamic Channel Selection:** Temporarily exclude channels with low activation values for specific inputs
- **Threshold-Based Decisions:** Use activation magnitude thresholds to determine which components to include
- **Adaptive Behavior:** Simple images with uniform regions allow many filters to be temporarily excluded
- **Latency Benefits:** Reduces computational workload during inference without permanently modifying architecture
- **Input Complexity Matching:** Allocates computational resources based on input difficulty for each specific case
- **Implementation Overhead:** Requires additional computation for activation analysis and pruning decisions

Runtime Sparsity Techniques

Adaptive Parameter Utilization

Runtime sparsity allows models to adjust their effective size based on current conditions:

- **Condition-Based Activation:** Enable or disable network components based on input characteristics or system state
- **Resource-Aware Scaling:** Adjust model complexity based on available computational budget or power constraints
- **Confidence-Based Refinement:** Use initial low-complexity inference; refine with full model if confidence insufficient
- **Temporal Adaptation:** Modify sparsity patterns over time based on workload characteristics or performance requirements
- **Multi-Model Variants:** Maintain multiple sparsity configurations optimized for different scenarios
- **Seamless Switching:** Transition between sparsity levels without interrupting inference pipeline

Size-Performance Trade-offs in Embedded Systems

Optimizing for Extreme Resource Constraints

Embedded deployment requires aggressive optimization within severe hardware limitations:

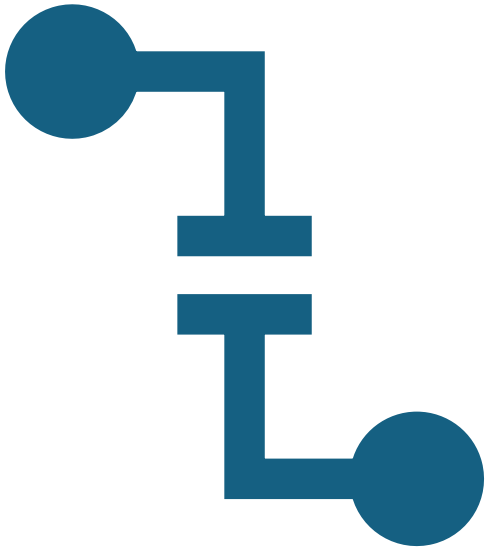
- **Flash Memory Constraints:** Models must fit in read-only storage alongside application code and OS; <1MB total for many microcontrollers
- **RAM Limitations:** Activation tensors compete with application memory; structured pruning reduces both parameter and activation memory
- **Computational Efficiency:** Limited processing power demands efficient execution; structured pruning provides predictable savings
- **Power Considerations:** 1 mW-and-below power range prevents use of anything beyond SRAM due to energy consumption
- **Real-world Example:** Bird detection requiring 250KB model, <100ms latency, >90% accuracy achieved through 60% parameter reduction plus quantization
- **System Context:** Pruning strategies must account for complete system including sensors, preprocessing, and user interface

Microcontroller Memory Architecture

Understanding Hardware Constraints

Microcontroller memory architecture creates specific constraints for model deployment:

- **Flash Storage:** Read-only memory for programs and constant data; written only during firmware upload; typically <1MB available
- **SRAM Technology:** Fast access with low power consumption but limited size; same technology used for CPU caches
- **Secondary Memory:** Some advanced microcontrollers offer DRAM but power consumption usually prohibitive for low-power applications
- **Power Budget Impact:** Enabling additional memory types can exceed power budget even if capacity is attractive
- **Memory Hierarchy:** Two-tier system with flash for storage and SRAM for computation; no complex cache hierarchies
- **Access Patterns:** Direct memory access without virtual memory management or complex caching strategies



Estimating Model Latency

Computational Complexity Analysis

Most neural network models spend majority of time in large matrix multiplications:

- **FLOP Calculation:** Work approximately equals input values × output values for each layer
- **Multiply-Add Operations:** Single instruction at machine code level often counts as two FLOPs
- **Quantized Operations:** 8-bit calculations sometimes still referred to as FLOPs despite no floating-point involvement
- **Layer-by-Layer Analysis:** Fully connected layer FLOPs = input vector size × output vector size
- **Convolutional Complexity:** Filter width × height × input channels × number of filters plus bias calculations
- **Proportional Performance:** 100 million FLOP model should run ~2× faster than 200 million FLOP version

FLOP Limitations and Reality

Beyond Theoretical Operation Counts

FLOPs provide rough estimates but don't capture all performance factors:

- **Software Optimization:** How well optimized the implementation is for particular layers affects actual latency
- **Memory Access Patterns:** Data movement and cache efficiency can dominate arithmetic operation time
- **Hardware Utilization:** Tensor cores, SIMD units, and specialized accelerators change FLOP-to-time relationships
- **Framework Overhead:** Function call overhead, memory allocation, and scheduling can add significant latency
- **Batch Size Effects:** Single-sample inference doesn't benefit from batching optimizations used in training
- **Practical Validation:** Always measure actual latency on target hardware rather than relying solely on FLOP estimates





Model Architecture Speed Optimization

Algorithmic Approaches to Latency Reduction

Architecture-level changes provide the biggest impact on neural network latency:

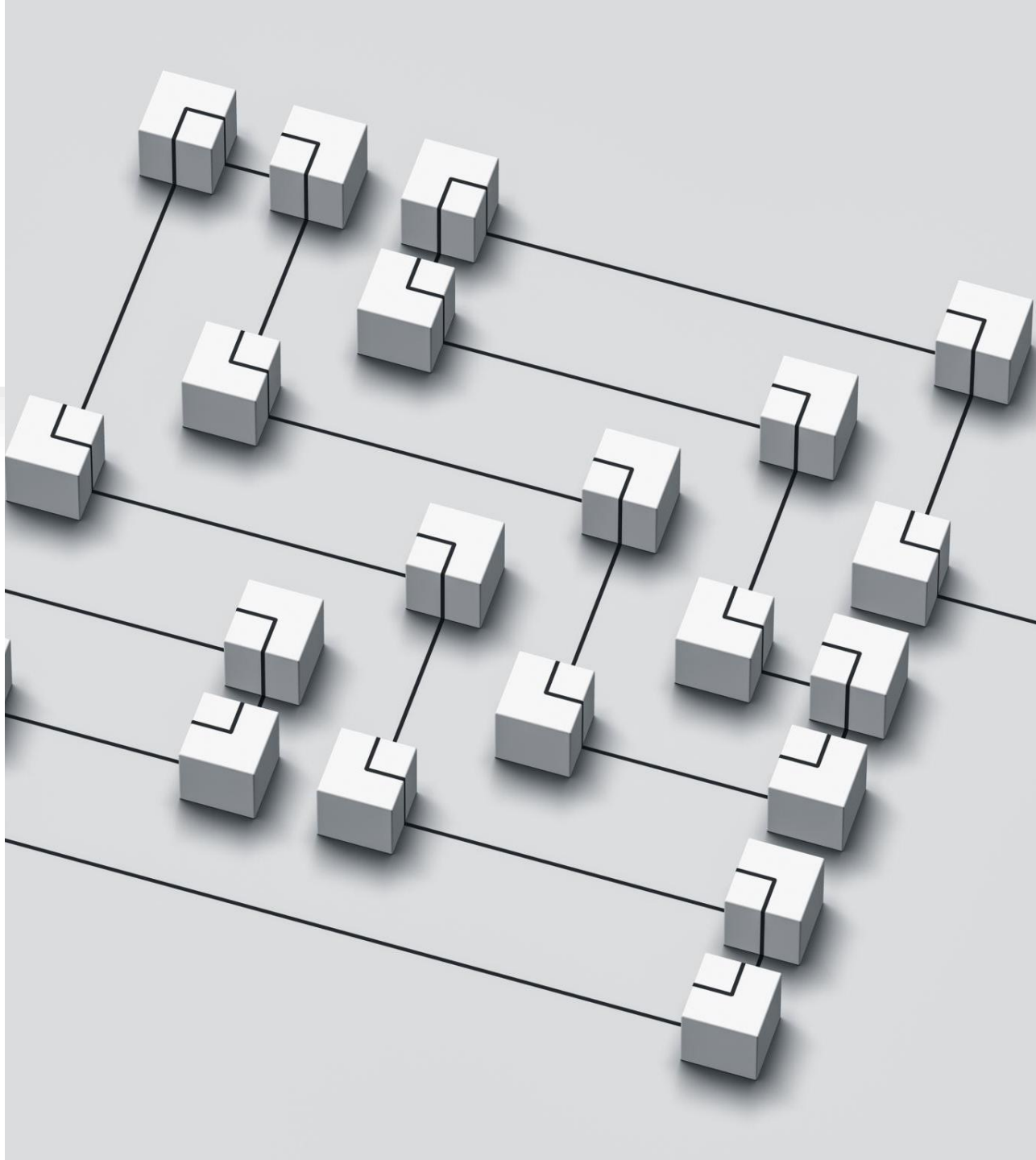
- **Calculation Reduction:** Creating models with fewer calculations speeds inference without code changes
- **Accuracy-Speed Trade-offs:** Starting with highly accurate model provides more room for trading accuracy for speed
- **Data Investment:** Improving training data helps throughout development, even for latency optimization tasks
- **Algorithmic Focus:** Better to eliminate work entirely rather than optimize existing computations
- **Model Swapping Advantages:** Easier to swap ML models than traditional algorithms; models are functional black boxes
- **Graceful Degradation:** Neural networks handle destructive changes well; can experiment with removing layers

Architecture Parameter Tuning

Model-Specific Optimization Opportunities

Many models have tunable parameters that affect computational requirements:

- **MobileNet Parameters:** Depthwise channel factor controls trade-off between accuracy and computational cost
- **Input Size Scaling:** Reducing input dimensions (e.g., image resolution) quadratically reduces computation for vision models
- **Layer Removal:** Systematically removing individual layers to observe accuracy and latency effects
- **Filter Reduction:** Reducing number of convolutional filters while maintaining architecture structure
- **Depth vs Width:** Trading network depth for width or vice versa based on hardware characteristics
- **Bottleneck Adjustments:** Modifying bottleneck layer sizes in architectures like ResNet and EfficientNet

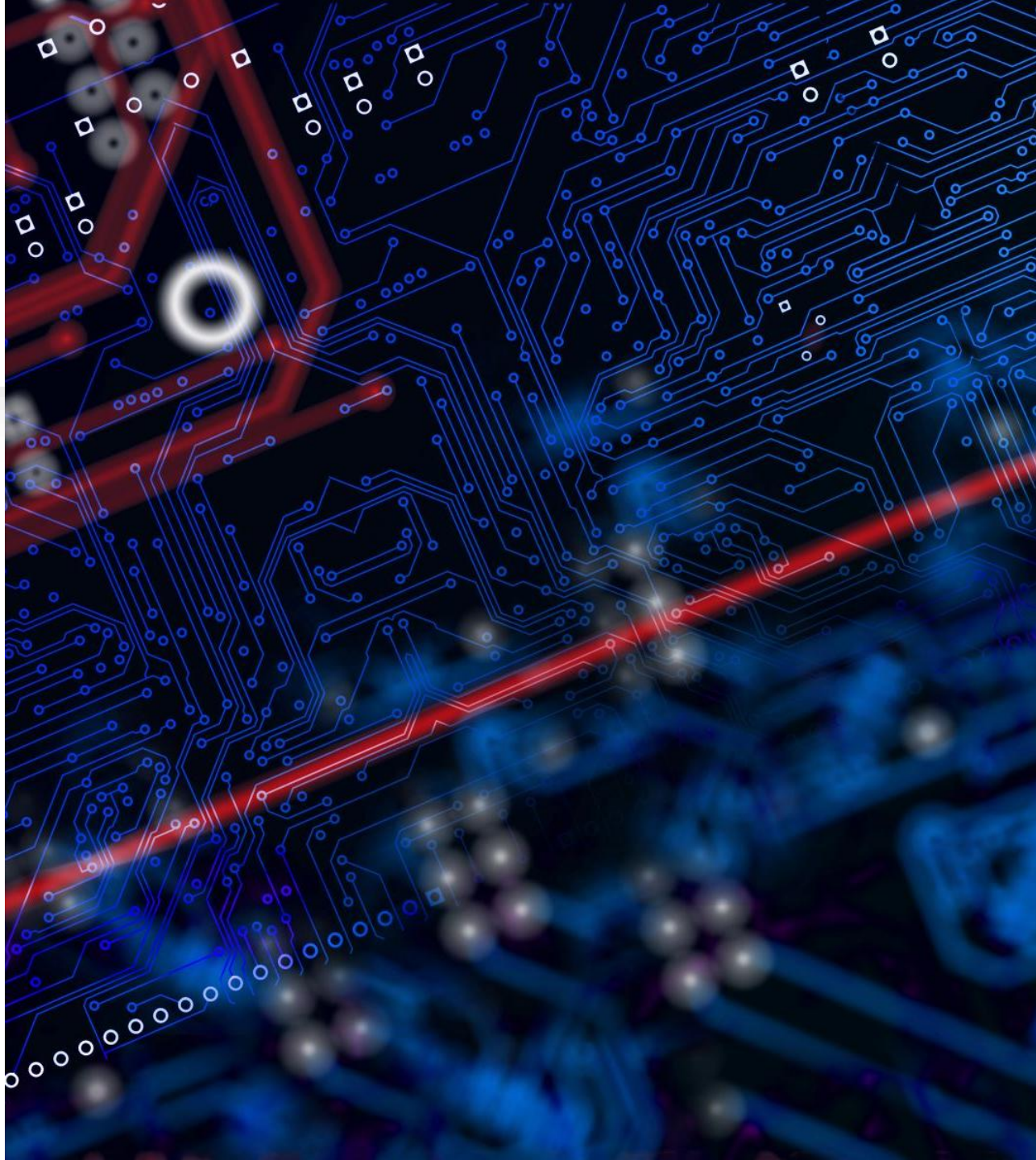


OpResolver for Code Size Control

Minimizing Framework Footprint

TensorFlow Lite supports over 100 operations, but models typically need only a subset:

- **AllOpsResolver Problem:** Convenient for prototyping but includes every supported operation, creating large binaries
- **Custom Resolver Creation:** Copy resolver files, rename appropriately, remove AddBuiltin() calls for unused operations
- **Automatic Dead Code Elimination:** Linkers remove unreferenced code; removing operations allows excluding implementations
- **MicroMutableOpResolver Alternative:** For few operations, directly create instance and add only needed registrations
- **Static Declaration Required:** Resolver objects must be static because interpreter can call them at any time
- **Significant Savings:** Expect 20-30 KB decrease when removing entire framework; proportional savings for individual operations



Custom OpResolver Implementation

Practical Code Size Reduction

Creating custom operation resolvers provides substantial memory savings:

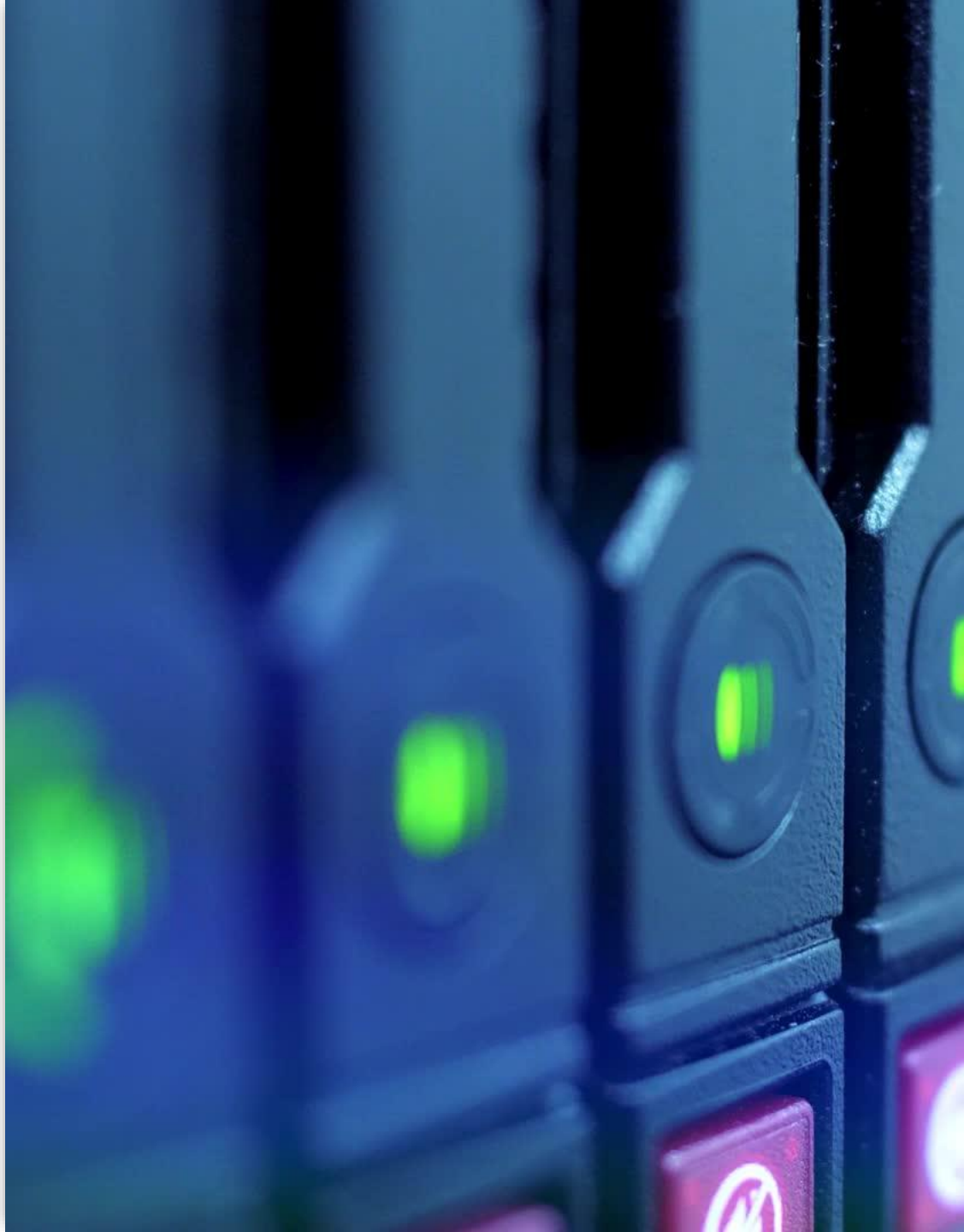
- **Operation Listing:** Use model inspection tools to determine which operations are actually used
- **Selective Registration:** Only register operations required by your specific model
- **Binary Size Impact:** Each operation implementation can take several kilobytes; removing unused ones adds up quickly
- **Maintenance Considerations:** Custom resolvers need updates when model changes or framework upgrades
- **Testing Requirements:** Ensure all model operations covered; missing registrations cause runtime errors
- **Framework Integration:** Custom resolvers work seamlessly with existing interpreter and inference pipeline

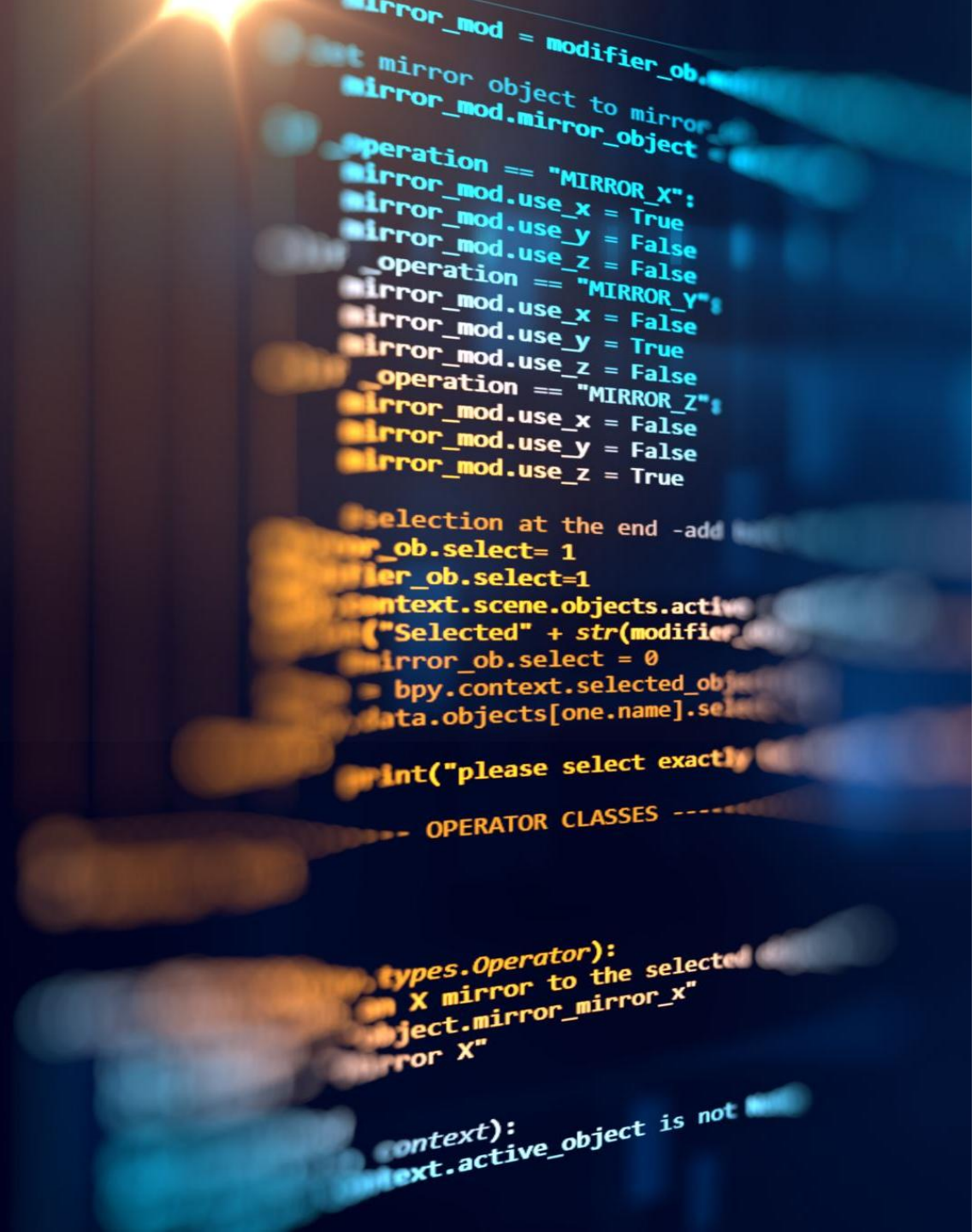
Measuring Code Size

Understanding Flash Memory Usage

Accurate code size measurement requires understanding the build and deployment process:

- **ELF vs Binary Files:** Linker output includes debugging information not transferred to device; measure actual flashed binary
- **Binary File Creation:** Use objcopy to create .bin file from ELF; this matches actual flash usage
- **Model Data Inclusion:** Binary size includes model data compiled as C array; subtract model size for code footprint
- **Linker Optimization:** Unused code automatically stripped; commenting out framework calls shows size reduction
- **Cross-Platform Tools:** Use toolchain-specific objdump for accurate analysis rather than desktop versions
- **Function-Level Analysis:** Use nm -S to inspect individual function and object sizes in intermediate files





Code Size Analysis Tools

Detailed Footprint Investigation

Several tools help understand where code size is consumed:

- **nm Command:** Shows symbol sizes in object files; useful for identifying large functions
- **objdump Disassembly:** Provides source-intermingled assembly to understand where code size comes from
- **Size Command:** Shows section sizes (text, data, BSS) for quick overview
- **Linker Maps:** Detailed breakdown of how much space each module and function consumes
- **Platform-Specific Tools:** Each toolchain provides specific utilities for code analysis
- **Binary Comparison:** Compare before/after binary sizes when making optimization changes

Framework Constants and Memory Tuning

Fine-Tuning Memory Usage

TensorFlow Lite for Microcontrollers uses hardcoded sizes to avoid dynamic allocation:

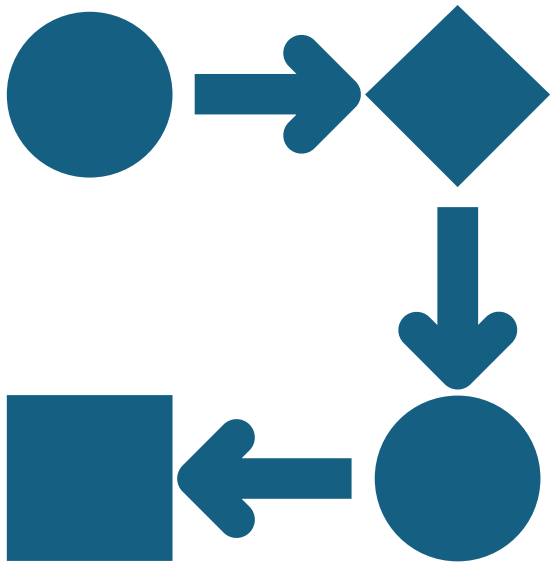
- **TFLITE_REGISTRATIONS_MAX:** Controls operation registration capacity; default 128 creates 4+ KB array
- **Typical Reduction:** Most applications use far fewer than 128 operations; reducing provides immediate RAM savings
- **Other Constants:** kStackDataAllocatorSize, arena size can be optimized based on actual usage
- **Memory Monitoring:** Instrument arena usage to determine optimal sizing; start large then trim
- **Conservative Approach:** Begin with oversized allocations, measure actual requirements, then optimize
- **Custom Tuning:** For extremely tight constraints, custom implementations may be necessary

Truly Tiny Models

Extreme Resource Optimization

For devices with kilobyte-level constraints, standard framework approaches insufficient:

- **Custom Code Requirement:** Must hand-tune everything for devices with couple kilobytes RAM/flash
- **Framework Value:** Still useful for training workflow and weight extraction even with custom inference
- **Verification Strategy:** Use existing framework to verify custom implementation results
- **Reference Implementation Benefits:** Portable, understandable, memory efficient starting points
- **Weight Extraction:** TFLite files provide structured format for accessing trained parameters
- **Validation Approach:** Compare custom results against framework version to ensure correctness



Integration with Other Optimizations

Combining Pruning with Complementary Techniques

Effective optimization strategies combine multiple techniques for maximum efficiency:

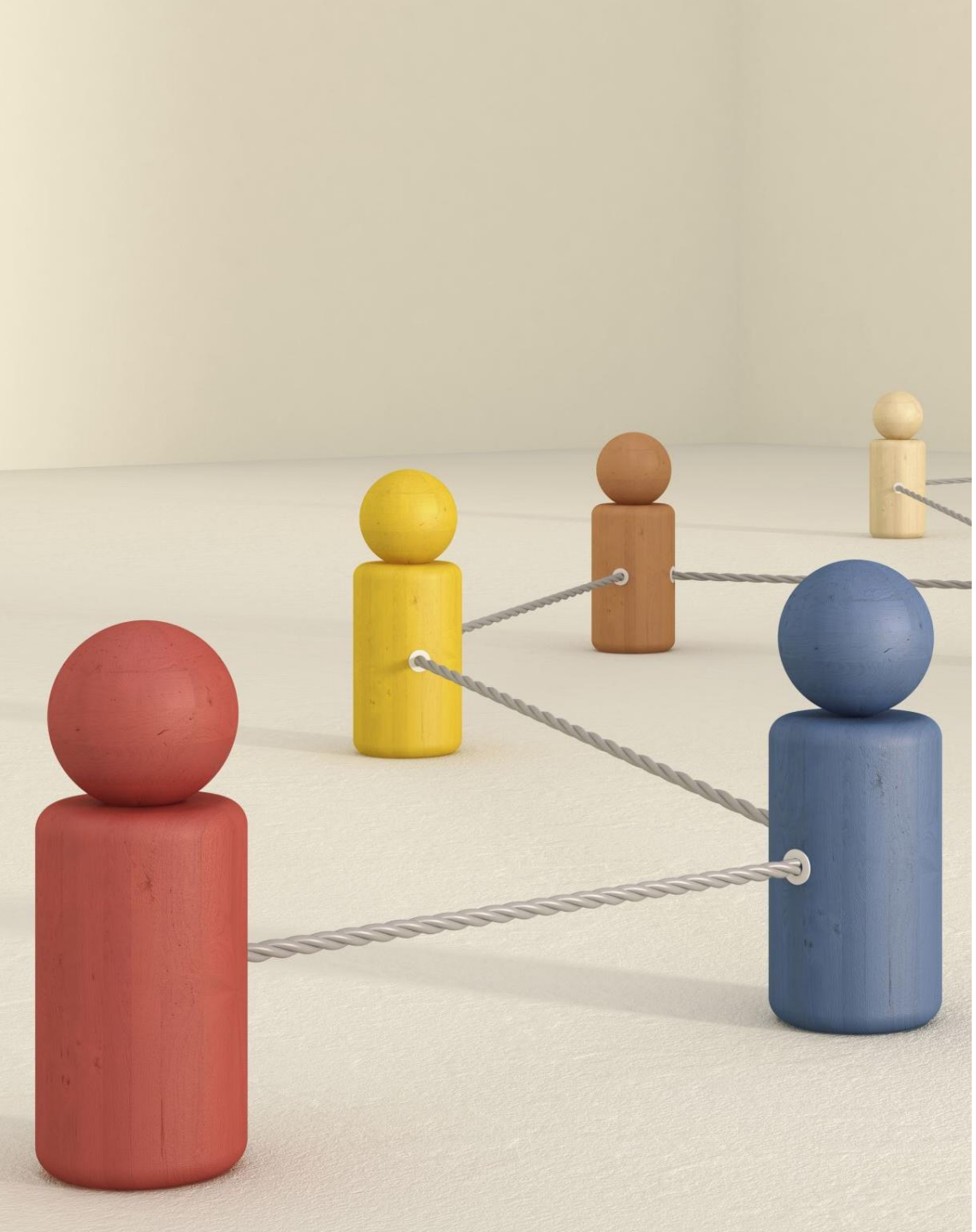
- **Pruning + Quantization:** Sequential application works best—prune first, then quantize remaining weights for 10× size reduction
- **Pruning + Knowledge Distillation:** Use original large model as teacher to guide training of pruned student model
- **Pruning + Architecture Search:** Design efficient architectures specifically for post-pruning optimization
- **Deployment Pipeline Integration:** Consider entire pipeline including model conversion and hardware-specific acceleration
- **Optimization Order:** Different sequences yield different results in accuracy, efficiency, and implementation complexity
- **MLOps Integration:** Modern pipelines increasingly include automated pruning as standard optimization stage

Quantization Integration

Combining Sparsity with Reduced Precision

Pruning and quantization provide complementary benefits when applied systematically:

- **Sequential Approach:** Prune parameters first to reduce count, then quantize remaining weights to reduce precision
- **Memory Multiplication:** 50% pruning + 8-bit quantization = 87.5% total memory reduction (×8 compression)
- **Accuracy Considerations:** Combined techniques may have compounding accuracy effects requiring careful validation
- **Framework Support:** TensorFlow Lite and PyTorch provide integrated workflows for both optimizations
- **Hardware Alignment:** Ensure target hardware supports chosen precision formats and sparsity patterns
- **Calibration Requirements:** Quantization may need recalibration after pruning changes activation distributions



Real-World Deployment Considerations

Bridging Theory and Practice

Successful pruning deployment requires attention to practical considerations:

- **Model Versioning:** Pruned models may require different update strategies; sparse models can be sensitive to weight updates
- **Calibration Shifts:** Pruning can shift model confidence distributions, requiring threshold recalibration
- **Edge Case Handling:** Aggressive pruning may reduce robustness to unusual inputs; comprehensive testing essential
- **Performance Monitoring:** Production systems need monitoring for accuracy degradation, especially with dynamic pruning
- **Framework Compatibility:** Ensure pruned models work with deployment frameworks and hardware acceleration libraries
- **Maintenance Burden:** Consider long-term maintenance implications when choosing pruning approaches

Production Deployment Pipeline

End-to-End Optimization Workflow

Deploying pruned models requires systematic workflow integration:

- **Development Stage:** Early export and operation listing to identify optimization opportunities
- **Training Integration:** Incorporate pruning into training pipeline with automated validation
- **Model Conversion:** Seamless export to deployment formats with optimization preservation
- **Hardware Validation:** Test actual performance on target hardware rather than theoretical estimates
- **A/B Testing:** Deploy pruned models alongside originals to measure real-world impact
- **Monitoring Infrastructure:** Continuous monitoring for accuracy degradation and performance regression

Validation and Testing Strategies

Ensuring Pruned Model Quality

Comprehensive validation becomes critical when pruning can introduce subtle degradations:

- **Distribution Validation:** Test pruned models on diverse datasets beyond original validation set
- **Hardware Benchmarking:** Measure actual performance on target hardware rather than theoretical estimates
- **Stress Testing:** Evaluate under challenging conditions specific to application domain
- **Production A/B Testing:** Deploy alongside original versions to measure real-world impact
- **Automated Pipelines:** Integrate validation into CI/CD to catch regressions early
- **Robustness Analysis:** Systematic evaluation under various input perturbations and environmental conditions

Framework and Tool Ecosystem

Practical Implementation Resources

Modern frameworks provide comprehensive pruning support:

PyTorch Ecosystem:
- torch.nn.utils.prune for built-in magnitude and structured pruning
- Third-party libraries for advanced techniques
- Integration with deployment tools like TorchScript

TensorFlow Ecosystem:
- Model Optimization Toolkit for training-time pruning
- TensorFlow Lite conversion with pruning preservation
- Seamless quantization workflow integration

ONNX Compatibility:
- Enables pruning in one framework, deployment in another
- Critical for diverse deployment environments

Specialized Tools:
- Neural Magic for production sparsity optimization
- Vendor-specific tools for mobile and embedded deployment

Advanced Pruning Research Directions

Emerging Techniques and Future Opportunities

Current research explores sophisticated approaches for improved pruning effectiveness:

• **Neural Architecture Search for Pruning:** Automatically discover architectures that maintain accuracy under aggressive pruning

• **Gradient-Based Pruning:** Uses second-order gradient information for more accurate parameter importance estimation

• **Structured Sparsity Patterns:** Explores specific sparsity patterns optimized for particular hardware architectures

• **Federated Pruning:** Distributed pruning across multiple devices with privacy constraints

• **Hardware-Software Co-design:** Joint optimization of pruning strategies and hardware accelerator design

• **Early Detection Methods:** Research into identifying winning subnetworks without full training cycles



Automated Pruning Systems

Next-Generation Optimization Tools

Emerging systems automate pruning decisions to reduce manual optimization effort:

- **AutoML Integration:** Automated model optimization pipelines that include pruning as standard component
- **Reinforcement Learning Approaches:** RL agents learn optimal pruning policies for specific hardware and accuracy targets
- **Differentiable Architecture Search:** End-to-end optimization of both architecture and sparsity patterns
- **Hardware-Aware Automation:** Systems that automatically adapt pruning strategies based on target hardware characteristics
- **Multi-Objective Optimization:** Balancing accuracy, latency, memory, and energy consumption simultaneously
- **Continuous Optimization:** Systems that adapt pruning during deployment based on changing requirements

Performance Benchmarking Best Practices

Measuring What Matters

Effective pruning evaluation requires careful measurement across multiple dimensions:

- **Model Size Metrics:** Parameter count reduction, actual memory footprint, storage compression ratio
- **Performance Metrics:** Inference latency on target hardware, memory bandwidth utilization, energy consumption
- **Accuracy Preservation:** Top-1 and top-5 accuracy, performance across input distributions, robustness to edge cases
- **Hardware Efficiency:** Actual speedup vs theoretical, cache miss rates, compatibility with inference engines
- **Practical Gaps:** Difference between theoretical and practical benefits often reveals optimization opportunities
- **Systematic Evaluation:** Consistent benchmarking across different pruning approaches and hardware platforms

Model Lifecycle Management

Maintaining Pruned Models in Production

Pruned models require special consideration throughout their operational lifecycle:

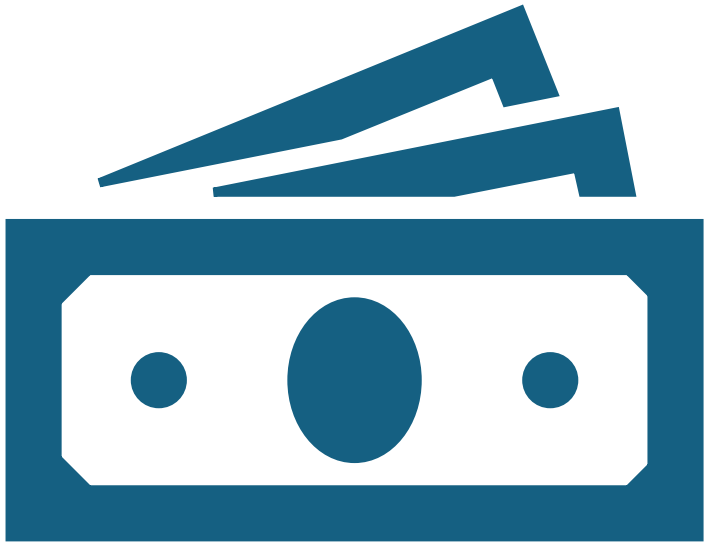
- **Version Control:** Track pruning configurations alongside model weights and architecture changes
- **Update Strategies:** Sparse models may require different update approaches than dense counterparts
- **Monitoring Requirements:** Additional metrics for sparsity levels, hardware utilization, and accuracy stability
- **Rollback Procedures:** Ability to revert to previous sparsity levels if performance degrades
- **Documentation Standards:** Clear recording of pruning decisions, validation results, and performance characteristics
- **Team Knowledge:** Ensure team understanding of pruning implications for maintenance and debugging

Edge Case Considerations

Handling Pruning Limitations

Several scenarios require special attention when deploying pruned models:

- **Distribution Shift:** Pruned models may be more sensitive to changes in input distribution
- **Adversarial Robustness:** Sparsity can affect model robustness to adversarial examples
- **Fine-Tuning Sensitivity:** Pruned models may respond differently to additional training
- **Hardware Dependencies:** Some pruning benefits only realized on specific hardware configurations
- **Framework Updates:** Pruning optimizations may not transfer across framework versions
- **Model Surgery:** Difficulty modifying pruned models for new requirements or capabilities

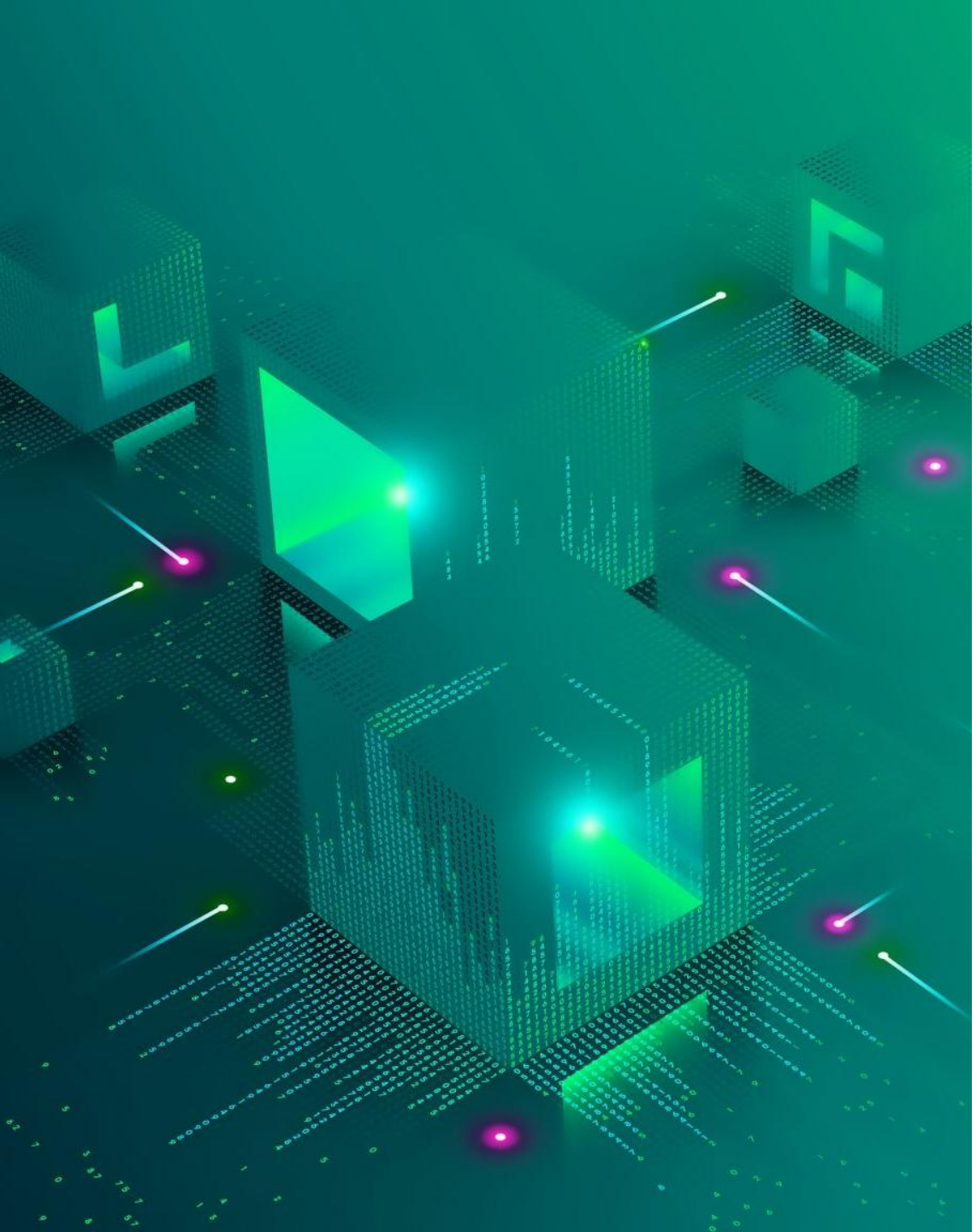


Cost-Benefit Analysis

Evaluating Pruning ROI

Pruning decisions should consider comprehensive cost-benefit analysis:

- **Development Costs:** Time and resources required for pruning implementation and validation
- **Deployment Benefits:** Reduced hardware requirements, improved latency, energy savings
- **Accuracy Trade-offs:** Cost of any accuracy degradation relative to application requirements
- **Maintenance Overhead:** Ongoing costs for managing and updating pruned models
- **Hardware Savings:** Quantifiable benefits from reduced memory and computational requirements
- **Time-to-Market:** Impact of pruning optimization on development timeline and deployment schedule



Future Hardware Trends

Evolving Support for Sparse Computation

Hardware landscape increasingly supports sparse computation paradigms:

- **Sparse Matrix Units:** Modern GPUs include dedicated hardware for sparse operations
- **Custom Accelerators:** AI chips designed specifically for neural networks with sparse computation capabilities
- **Structured Sparsity Hardware:** Optimized support for specific sparsity patterns like block-sparse operations
- **Software-Hardware Co-design:** Closer integration between pruning algorithms and hardware capabilities
- **Industry Standards:** Emerging standards for sparse computation across different hardware vendors
- **Performance Trajectory:** Improving gap between theoretical and practical sparse computation benefits

Summary - Week 9 Key Takeaways

Pruning for Practical Deployment

Core Pruning Concepts:
• Mathematical foundation: Constrained optimization with practical relaxations
• Unstructured vs structured trade-offs: Flexibility versus hardware compatibility
• Strategic approaches: Iterative refinement versus aggressive one-shot optimization
• Dynamic adaptation: Runtime optimization for varying computational demands

Implementation Principles:
• Hardware-aware strategy selection based on deployment constraints
• Framework integration leveraging existing optimization toolkits
• Comprehensive validation ensuring robust performance across diverse conditions
• Systematic combination with complementary optimization techniques

Practical Impact:
• Enable deployment of sophisticated models on resource-constrained devices
• Achieve order-of-magnitude improvements in storage and computational efficiency
• Maintain high accuracy through careful strategy selection and validation
• Bridge the gap between research models and production deployment requirements

Programming Assignment 4 — Model Compression and Optimization Pipeline (Due: Week 12)

Objective

- Build a compression pipeline combining pruning, quantization, knowledge distillation, and NAS to produce efficient models for resource-constrained deployment.
- Focus on optimization techniques and their interactions, not full system deployment.

Baseline & Targets

- Dataset: CIFAR-100
- Baseline accuracy: >75% on test set
- Compression goal: 10× size reduction with <5% accuracy loss

What You'll Implement

- Pruning: Unstructured (magnitude) + structured (filter/channel), gradual vs one-shot, sensitivity analysis, lottery ticket test.
- Quantization: Mixed-bit, PTQ vs QAT, INT4/binary exploration, dynamic vs static, error propagation.
- Distillation: Temperature tuning, progressive (multi-stage), attention transfer, feature matching, self/ensemble distillation.
- Interaction Analysis: Pairwise and combined pipelines; ordering effects; Pareto frontiers (accuracy vs size/latency).

Evaluation & Tools

- Metrics: Accuracy, model size (MB), latency (ms), memory (MB), FLOPs reduction, compression ratio, energy (if possible).
- Visualizations: Pareto frontiers, compression heatmaps, convergence plots, interaction matrices, radar charts.

Deliverables

- Code: baseline_model.py, part1_pruning.py, part2_quantization.py, part3_distillation.py, part4_interaction_analysis.py, compression_evaluator.py, visualization_tools.py, application_scenarios.py, requirements.txt, README.md
- Models: baseline_model.keras (>75% acc), compressed_models/, checkpoints/
- Analysis: compression_analysis_report.pdf (3–4 pp), compression_guidelines.md, performance_benchmarks.xlsx, interaction_analysis_summary.pdf
- Visuals: pareto_frontiers.png, compression_heatmaps.png, convergence_plots.png, interaction_matrices.png

Looking Ahead - Week 10 Preview

Quantization: Precision Optimization

Next Week's Focus:

- Numerical precision optimization across the deployment spectrum
- Energy implications of different precision formats
- Hardware support for quantization operations
- Integration with pruning for maximum efficiency

Connection to Pruning:

- Complementary optimization techniques often applied sequentially
- Combined approaches achieve greater efficiency than either technique alone
- Validation strategies must account for interaction effects
- Framework support enables integrated optimization workflows

Required Readings for Week 10:

- Book 1 – Volume 1: Model Compression
- Book 1 – Volume 1: Hardware Acceleration
- Book 1 – Volume 1: Benchmarking
- Book 1 – Volume 1: Model Serving
- Book 2: Chapter 16: Optimizing Energy Usage (precision-energy relationships)
- Book 2: Chapter 17: Optimizing Model and Binary Size (quantization sections)